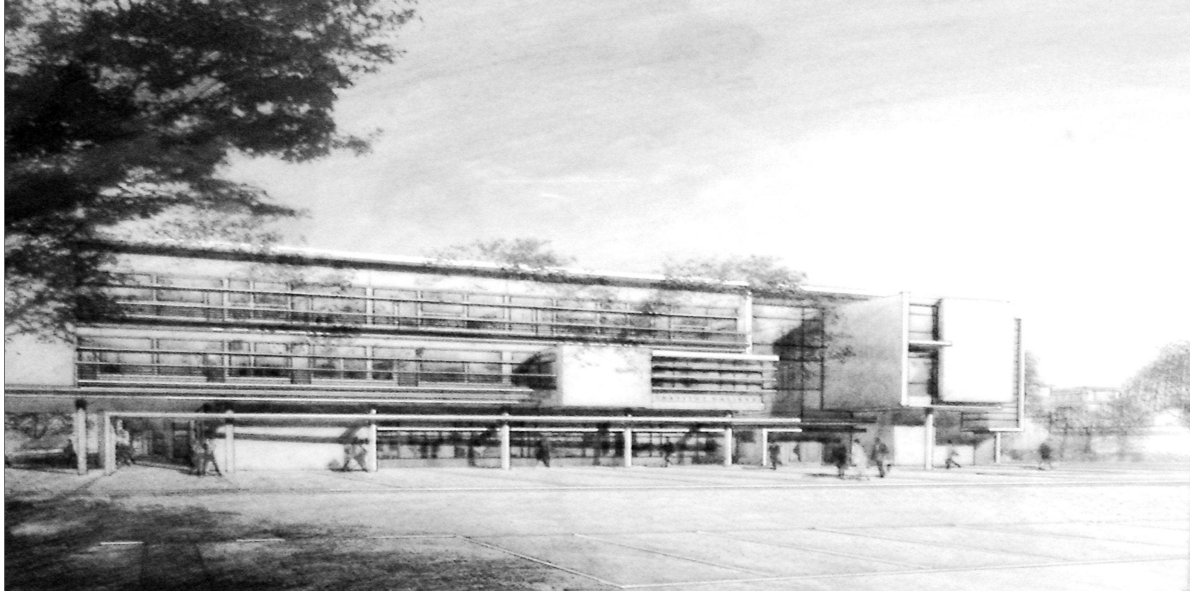




Microcontrôleurs PIC

TRAVAUX PRATIQUES MICROCONTRÔLEURS

THEMES ABORDES EN TP

TP 0 : PRISE EN MAIN DE LA CHAÎNE DE DEVELOPPEMENT	03
TP 1 : MODULES DE TEMPO ET ENTREES/SORTIES LOGIQUES	26
TP 2 : PRISE EN MAIN DU LANGAGE C	44
TP 3 : PROGRAMMATION DES INTERFACES EN LANGAGE C	58

LES TRAVAUX PRATIQUES DOIVENT OBLIGATOIREMENT ETRE PREPARES
AVANT LA SEANCE EN LABO :
CE POINT SERA SYSTEMATIQUEMENT CONTRÔLE PAR L'ENSEIGNANT.

THIEBAUT Bernard

Remarques générales sur les séances de travaux pratiques

Ces séances permettent de mettre en pratique des notions plus théoriques présentées en cours.

*En début du cycle de TP, les étudiants **se regrouperont en binômes**. Les regroupements seront définitifs et valables pour tout le cycle. (exceptionnellement en cas d'absence dans un binôme, l'étudiant présent pourra se joindre à un groupe de deux étudiants complet mais il faudra l'indiquer sur le compte-rendu correspondant).*

Dans des disciplines comme l'électronique programmable, **rien ne vaut la manipulation réelle** pour s'assurer que les principales notions ont bien été comprises. Dans certains cas même, les concepts qui n'auraient pas été assimilés en cours peuvent apparaître plus clairs et plus compréhensibles lorsqu'on "**met les mains dans le cambouis**".

Cela veut dire aussi qu'une deuxième lecture du cours sera indispensable et très profitable sous cet éclairage nouveau fourni par la manipulation.

On insistera beaucoup sur la notion de **programmation modulaire** et on espère que, au fur et à mesure de leur avancement, les étudiants seront convaincus de son intérêt.

Il reste qu'il est **indispensable que le cours soit bien connu** avant d'aborder la séance. On rencontre malheureusement trop souvent des étudiants qui visiblement n'ont pas « bûché » leurs cours : ceux-là passeront à coup sûr à côté de cette discipline.

Il faudra également **penser à apporter ses différents polycopiés** (cours, système de développement, TP ...) car dans cette discipline, on " n'invente rien " mais on est souvent amenés à exploiter correctement de la documentation.

De plus, une grande partie de la documentation constructeur est également disponible (mais souvent en **anglais**) sur les PC des salles de manip.

Il est également **obligatoire de posséder un support informatique** pour sauvegarder son travail d'une séance à l'autre. (certaines parties peuvent en effet resservir par la suite, dans cette même série de TP).

Toutes les séances ayant lieu sur ordinateur, la première chose à faire en arrivant devant une machine **est de créer son propre répertoire dans le dossier Etudiants/Formation :**

Toute intrusion dans un autre répertoire que le sien pour y stocker ses fichiers de travail sera sanctionnée par l'enseignant par un malus de - 2 points sur la note finale de TP !!!

1. Préparation de la séance.

Les séances de travaux pratiques permettent de vérifier expérimentalement certaines des notions vues en cours. Il est donc **indispensable** que le cours soit parfaitement appris avant d'aborder la séance. Pour chacune de ces séances, l'étudiant dispose d'un sujet qui le guide dans son travail et lui demande de répondre à un certain nombre de questions et/ou de réaliser un programme.

Une grande partie de ce travail sera à faire avant la séance de manipulation.

Sur les pages de garde de chaque TP sont indiqués les éléments théoriques à regarder et/ou les questions théoriques à préparer avant chaque séance.

En début de séance, l'enseignant fera un rapide « tour de table » pour vérifier et évaluer le degré de préparation de chaque étudiant/binôme.

2. Travail en salle de TP.

Pour vérifier le bon avancement du travail en séance, des appels à l'enseignant sont régulièrement demandés pour que ce dernier contrôle le bon fonctionnement et la validité des programmes réalisés, vérifie si les tests ont été conduits en toute rigueur et si les points abordés ont bien été compris en posant quelques petites questions en rapport avec les manipulations effectuées et les résultats obtenus.

Si l'enseignant accepte que les binômes puissent échanger quelques « tuyaux » ou affiner entre eux quelques points plus délicats, les « transferts complets » de programmes seront sanctionnés, même s'ils ne sont « pris sur le fait » : l'enseignant est tout à fait capable de voir si un programme a bien été écrit (et compris) par un binôme mais il sait aussi détecter si un programme a été intégralement « pompé » en posant quelques questions bien ciblées.

3. Compte rendu.

Chaque binôme devra effectuer un compte rendu précis de son travail qu'il faudra remettre à l'enseignant. Sa rédaction fera apparaître chaque fois toutes les étapes de l'analyse des problèmes posés en utilisant les outils descriptifs appropriés :

organigrammes, justifications théoriques, listings **commentés** ...

La présentation sera **claire, aérée et soignée** et la rédaction sera faite dans un **français grammaticalement correct**. Dès que celle-ci s'impose, une **conclusion clairement exposée** fera la synthèse des différentes étapes du travail demandé.

La note finale du TP tiendra compte des 3 points précédents :

- de la préparation faite avant la séance,**
- du travail en séance et des résultats obtenus,**
- du compte rendu.**

La **non remise** du compte-rendu, l'**absence non justifiée** à une séance entraînera l'attribution d'une **note nulle**.

TP N° 0

Objectifs principaux :

Prise en main de la chaîne de développement
Construction d'un projet : assemblage et linkage
Utilisation du simulateur de MPLAB
Premiers pas dans le « débogage » de programmes
Instructions et directives de base

*Exceptionnellement pas de préparation particulière pour ce TP N°0,
mais bien connaître son cours*

Enseignant : THIEBAUT Bernard

Ce document n'est pas aussi exhaustif que les documentations constructeurs (sur les matériels et/ou les logiciels) et en cas de recherche d'informations plus précises ou plus pointues, il faudra se référer soit aux photocopiés de cours et/ou de résumés, soit aux documentations techniques, tous ces documents étant présents sur les PC des TP.

I. INTRODUCTION GENERALE.

La méthode utilisée est toujours un peu la même, quelque soit le système cible sur lequel on développe une application. Nous allons en donner les principales étapes et ce sera cette méthode qui sera utilisée dans les TP (ou pour n'importe quel projet ...).

La figure 0.1. de la page suivante résume d'ailleurs ces différentes étapes et les liens qui existent entre elles.

1. On prend connaissance du cahier des charges, ici de l'énoncé de l'exercice et on en effectue une analyse fonctionnelle (organigramme, découpage en bouts plus simples (modules), utilisation de pseudo langage ou de pseudo code pour écrire son programme...). C'est la **phase d'analyse**.

2. On utilise un PC (ou une station de travail), appelé aussi **PC hôte**, comportant tout l'environnement de travail pour gérer des projets et pour développer des programmes sources soit en assembleur natif du microcontrôleur de la **carte cible**, soit directement en langage C. Ici l'outil de développement est **MPLAB**. C'est la **phase d'écriture du source**.

3. Ces sources sont ensuite compilés puis linkés en fonction de la cartographie mémoire de la carte cible. C'est la **phase de compilation/linkage** ou **phase de construction**.

4. et 5. Le compilateur/linker détecte des erreurs (de syntaxe pure ou de linkage) qu'il faut corriger. Suite à cette étape, rien ne garantit encore que votre programme fonctionne comme c'était prévu. La seule chose qu'on peut dire c'est que votre exécutable est compréhensible par la carte cible. C'est la **phase de correction des erreurs de syntaxe/linkage**.

6. Il faut ensuite envoyer ce fichier exécutable sur l'outil de « débuggage » **ICDx** (ou alors utiliser le **simulateur** de **MPLAB**). C'est la **phase de téléchargement**.

7. Une fois le fichier implanté sur le débogueur (ou le simulateur), il ne reste plus qu'à le mettre au point en l'exécutant (soit en mode normal, soit le plus souvent en pas à pas). C'est le **début de la phase de test**.

8. En cas de problème, et c'est toujours le cas, il faut corriger le source, le recompiler, le télécharger à nouveau puis le re tester (refaire les phases 2, 3, 4, 5, 6, 7 et 8 tant que des problèmes subsistent...). C'est la **phase de mise au point** ou de « débuggage » qui peut être plus ou moins longue, suivant la complexité et la taille des programmes réalisés.

9. Jusqu'à l'obtention du résultat escompté (respect du cahier des charges). Plusieurs équipes peuvent d'ailleurs se réunir dans cette étape, pour coupler leurs résultats en vue de réaliser un projet plus conséquent. Puis il faut implanter l'application finale dans la mémoire programme **EEPROM** de la cible pour la rendre autonome et couper « le cordon ombilical » avec les outils de développement. C'est la **phase d'intégration finale en laboratoire**.

10. Qui est suivi dans l'industrie de la livraison au client, qu'il faut convaincre et satisfaire... C'est la phase finale **d'intégration sur site** ou **de recette**.

11. Suivi de la **facturation** et du **paiement** !!!

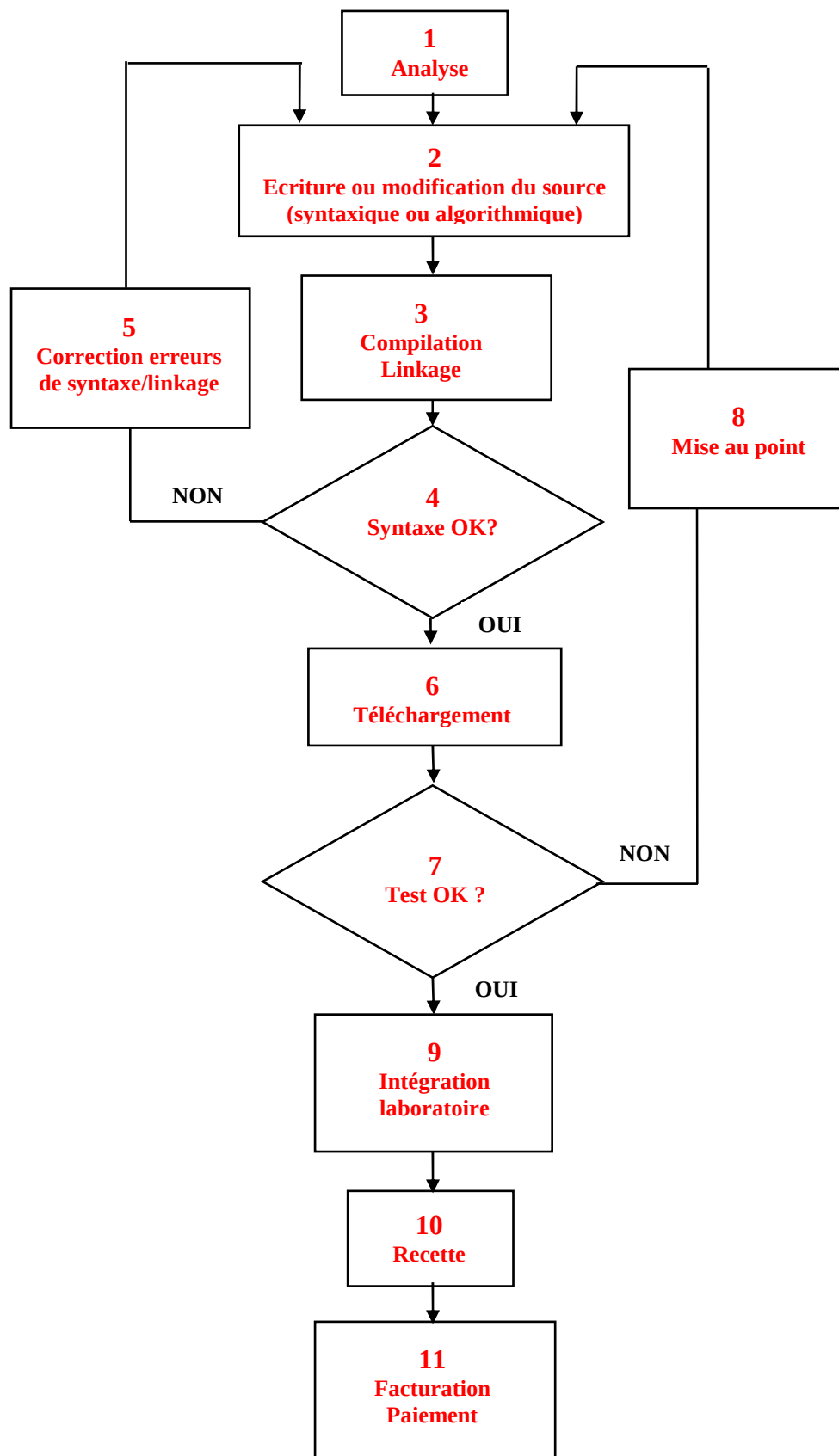


Figure 0.1.

II. PREPARATION DE SON ENVIRONNEMENT DE TRAVAIL

Dans le dossier **Etudiant**, ouvrir le répertoire correspondant à votre formation.

Vous y trouverez un dossier **d_pic** qui regroupe :

① un répertoire **documentations** contenant des informations utiles pour ces TP avec les sous répertoires suivants :

- **docs asm** : la documentation sur l'assembleur **MPASM**,
- **docs C**: la documentation sur le **compilateur C** utilisé,
- **docs carte PICDEM 2 PLUS** : la documentation sur la carte **cible**,
- **docs LCD ASCII** : plusieurs documentations sur le **LCD** implanté sur la carte cible,
- **docs MPLAB** : la documentation sur l'outil IDE **MPLAB**,
- **docs pic** : le data sheet du **PIC 16F877** et **16F877A** et de la famille **MIDRANGE**,
- **docs clavier matriciel** : une documentation sur les claviers matriciels,
- **docs emetteur/recepteur ultra sons** : une documentation sur les modules ultra sons,
- **docs LCD graphique**: une documentation sur les LCD graphiques,
- **docs moteur pas a pas**: une documentation sur les interfaces de puissance,
- **docs GPS**: une documentation sur une interface de type GPS,
- **docs voice direct**: une documentation sur un module de reconnaissance vocale,
- **docs servomoteur**: une documentation sur les servomoteurs,
- **docs I2C**: une documentation sur les composants compatibles I²C...

② un répertoire **prog_asm** avec des sous répertoires de projets écrits en **assembleur**, utiles pour les tutoriaux en assembleur et pour les exercices des TP,

③ un répertoire **prog_c** avec des sous répertoires de projets écrits en **langage C** utiles pour le tutorial en C et les exercices des TP,

④ un répertoire **fichiers HEX OK** comportant des exécutables qui vous donneront parfois une idée sur ce qu'on vous demande de faire en TP, un peu comme un exercice terminé... (certains exécutables proviennent de sources écrits en langage assembleur et d'autres de sources écrits en langage C).

⑤ un répertoire **cours TP BT** avec la version pdf de mon **cours** et du **TP** sur le PIC, ainsi que les différents résumés (cours et informations TP). Des informations sur des éventuels projets et extensions sont également disponibles sous la forme d'un résumé de projets.

Important : il faut **laisser intègre** cet original du dossier **d_pic**.

Commencer par créer votre propre répertoire avec votre nom ou vos initiales dans le dossier **Etudiant/Formation**.

Recopier (**et non glisser**) ensuite le dossier **d_pic** précédent dans votre propre répertoire.

Il faut **impérativement** laisser l'**original de d_pic intègre** dans **Etudiant/Formation**.

Vous ne pourrez le modifier que lorsqu'il aura été recopié dans votre propre répertoire, c'est-à-dire lorsqu'il vous « **appartiendra** » ...

Toute cette procédure présente au moins 4 avantages majeurs :

- ① elle permet d'éviter d'écrire n'importe où sur le disque et facilite le travail de ceux qui s'occupent de la maintenance des ordinateurs,
- ② elle assure un minimum de structuration de son travail,
- ③ de plus, en fin de séance, vous pourrez plus facilement sauvegarder **tout votre travail** en sauvant sur votre support le **contenu complet de votre répertoire**,
- ④ et enfin, vous aurez également sur ce support tous les éléments pour la préparation des TP.

Toute intrusion dans un autre répertoire que le sien pour y stocker des fichiers sera sanctionnée par l'enseignant par un malus de - 2 points sur la note finale de TP.

(cf. page de remarques générales sur les séances de travaux pratiques)

**A la fin de chaque séance,
les répertoires personnels des étudiants
pourront être supprimés
par l'enseignant ou les techniciens des labos !!!**

**Il est donc important que ce travail de sauvegarde
soit fait avant par vous !!!**

III. DEUX PETITS « TUTORIALS »

Une façon classique de présenter de nouveaux logiciels à utiliser consiste à réaliser ce qu'on appelle un "tutorial" en anglais, c'est-à-dire une prise en main guidée sur un exemple concret complet.

C'est ce que l'on va faire maintenant.

On vous demande donc de réaliser toutes les étapes de ces deux petits tutoriaux, en réfléchissant bien à ce que vous faites à chaque étape et pourquoi on vous demande de le faire pour bien comprendre les différentes fonctionnalités de MPLAB.

Répondez également aux petites questions qui vous sont posées.

Lisez bien aussi le texte du TP qui est assez détaillé ...

Ainsi, en assimilant correctement cette méthode de travail, vous deviendrez bien plus que de simples exécutants chargés de presser des boutons !!!

En respectant ces consignes, vous y gagnerez en capacités d'autonomie, en intérêt pour la matière et vous progresserez à coup sûr dans cette discipline.

Dans ces deux tutoriaux, nous allons utiliser la méthode **par simulation**, qui ne nécessite aucun matériel supplémentaire, si ce n'est un ordinateur perso.

Ainsi, vous pourrez dès à présent, télécharger l'outil de développement **MPLAB** sur votre PC perso pour pouvoir travailler et progresser en autonomie chez vous.

Voir pages 2 à 4 du fichier Résumé du cours pendant TP PIC.

**A la demande des étudiants,
l'enseignant pourra faire une présentation orale des trois méthodes de travail possibles.**

III.1. TUTORIAL N°1

Quelques éléments préliminaires théoriques avant de démarrer le tutorial 1:

Voir pages 5 à 12 du fichier **Résumé du cours pendant TP PIC**.

A la demande des étudiants,
l'enseignant pourra faire une présentation orale de ce tutorial 1.

1. Lancement de l'outil de développement.

Ouvrir le dossier **PIC** du bureau de Windows et lancer **MPLAB** (en principe version **v8.50**).

L'écran suivant (cf. figure 0.2.) s'ouvre alors :

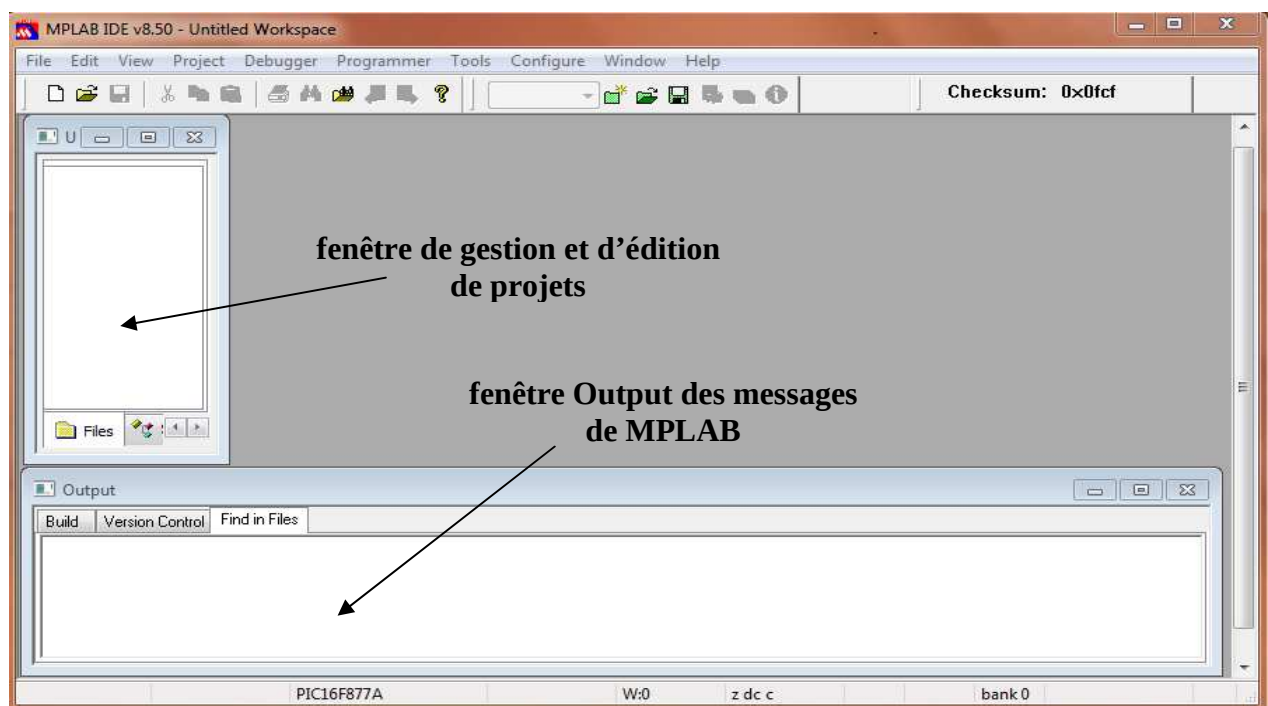


Figure 0.2.

2. Ouverture d'un projet existant et examen de ses différents fichiers constitutifs.

Dans le menu de **MPLAB**, faire :

Project Open

et aller rechercher dans **votre** répertoire de travail le projet **tutorial 1**, présent dans le sous-dossier **d_pic/prog_asm/tutorial 1**.

La **notion de projet** est plus large et plus complète que celle de **fichier source** : un projet est un ensemble de fichiers sources (programmes sources, fichiers d'inclusion, fichiers scripts, fichiers objet, fichiers issus de bibliothèques...) constituant une application complète.

Examiner dans la fenêtre de gestion et d'édition de projet, la structure de **tutorial 1.mcp** :

vous y verrez, attaché au projet, le fichier **tutorial 1.asm** dans les **Source Files** et le fichier **P16F877A.INC** dans les **Other Files**.

*(notons que ces deux fichiers se trouvent dans le répertoire **tutorial 1** du dossier **prog_asm** de votre répertoire de travail)*

Ouvrez (s'il ne l'est pas déjà) le fichier **tutorial 1.asm** en double cliquant sur lui dans la fenêtre de projets.

Parcourez **rapidement** ce fichier **tutorial 1.asm** en repérant notamment la zone de déclaration des variables puis celle du code.

Ouvrez (s'il ne l'est pas déjà) le fichier **P16F877A.INC** en double cliquant sur lui dans la fenêtre de projets.

Expliquer son rôle et indiquez où on y fait référence dans le fichier **tutorial 1.asm**.

Un conseil : bien gérer **MPLAB** en ce qui concerne les ouvertures des différentes fenêtres et de leur positionnement à l'écran : vous vous y retrouverez plus facilement et vous y gagnerez en confort de travail.

3. Construction (assemblage/linkage) du projet tutorial 1.

Comme on l'a vu dans le cours, il est nécessaire de transformer les fichiers source d'un projet dans le langage compréhensible par le PIC, c'est-à-dire en suites binaires de 1 et de 0 correspondantes. C'est le rôle du couple **assembleur** ($\equiv$ compilateur) / **linker**.

Dans le menu de **MPLAB**, faire :

Project Build All.

Si tout se passe bien (et ça doit être le cas ici) vous devrez obtenir dans la fenêtre **Output** du bas de votre écran :

BUILD SUCCEEDED

Quand vous construirez vos propres projets toutes les erreurs et « warnings » seront reportées dans cette fenêtre, comme par exemple :

- impossibilité d'accéder à certains fichiers de votre projet (erreurs de linkage),
- erreurs de syntaxe (erreurs d'assemblage) ...

Le constructeur de projet a fabriqué un certain nombre de fichiers, portant tous le même nom que le projet, mais avec des **extensions différentes**, comme indiqué sur la figure 0.3. suivante (où d'ailleurs tous les fichiers créés n'ont pas été représentés par souci de simplification) :

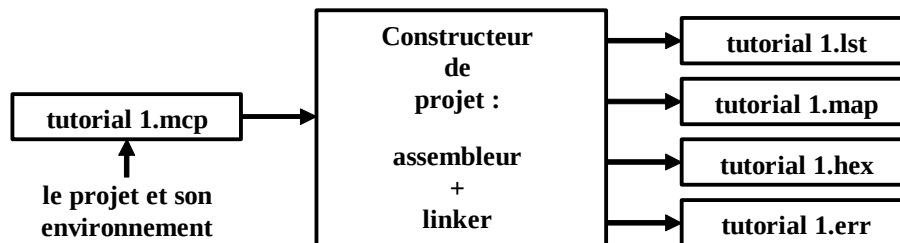


Figure 0.3.

Pour s'en convaincre, il suffit, dans le menu de **MPLAB**, de faire :

File Open et All Files

pour voir apparaître la fenêtre de la figure 0.4. ci-dessous :

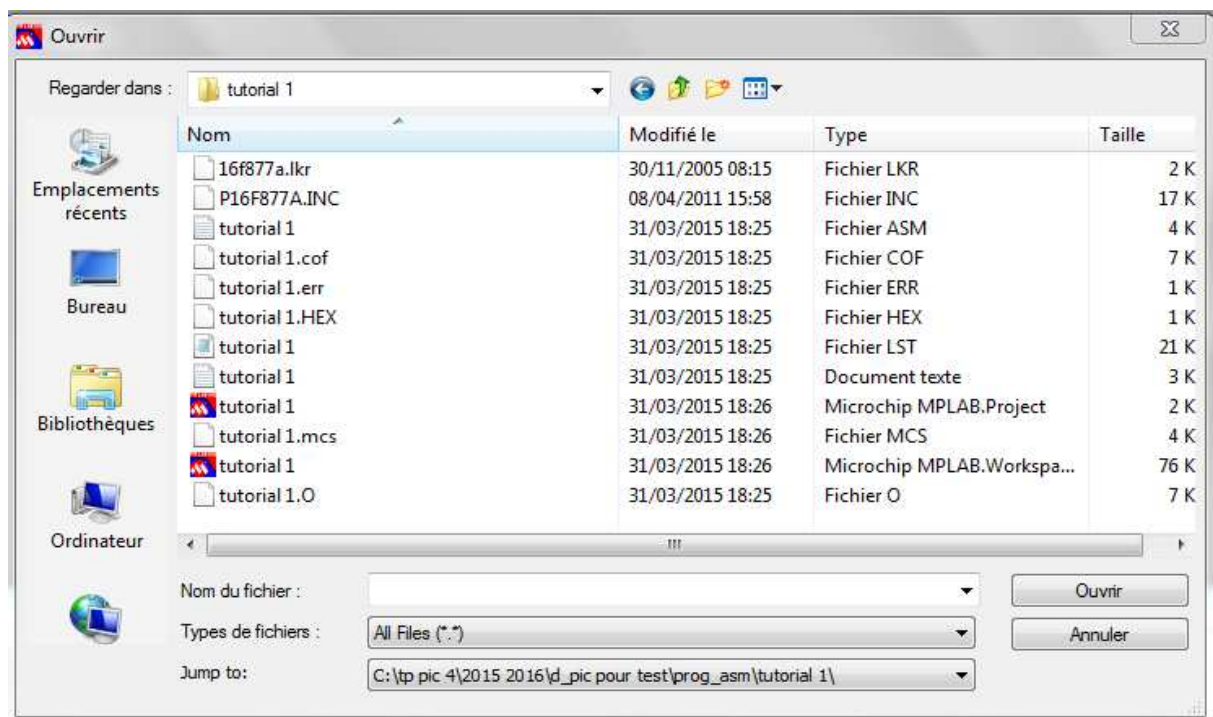


Figure 0.4.

Très brièvement, et vous pouvez éditer rapidement (non obligatoire) ces fichiers pour le vérifier :

- **tutorial 1.lst** : fait le lien entre l'implantation mémoire et le source assembleur,
- **tutorial 1.map** : donne la correspondance entre les noms symboliques et les adresses,
- **tutorial 1.hex** : fichier binaire utilisé pour la programmation du PIC,
- **tutorial 1.err** : reprend les erreurs et les « warnings » de la construction du projet.

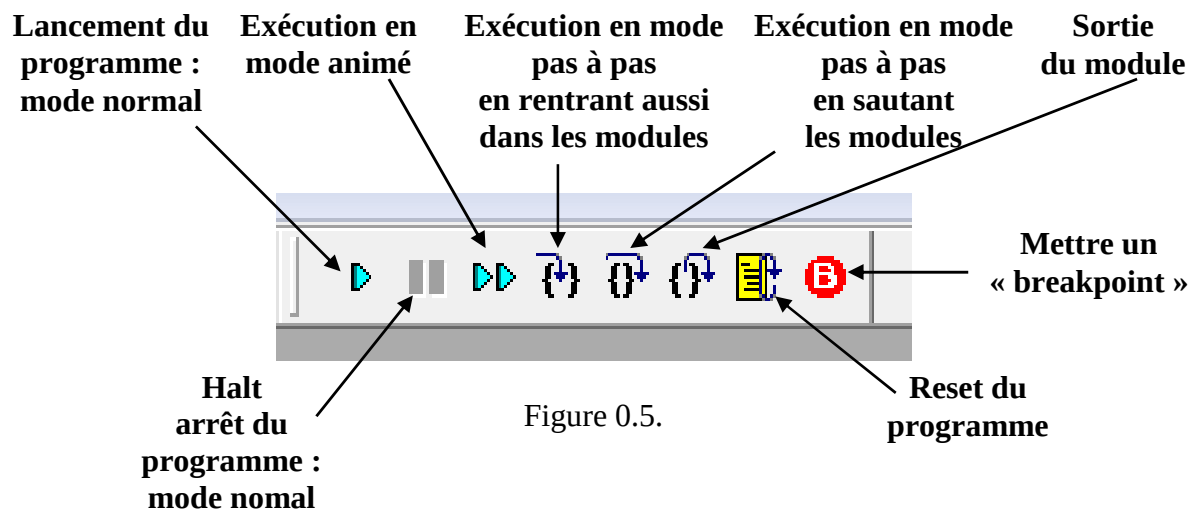
4. Choix de l'outil de débogage.

Dans le menu de **MPLAB**, faire :

Debugger Select Tool MPLABSIM

Car, c'est le simulateur **MPLABSIM** qui sera utilisé ici.

Apparaissent alors, dans le menu, les icônes suivants, donnés sur la figure 0.5. et utiles pour la mise au point, comme on le verra plus loin (pour le moment, essayez juste de les comprendre)



5. Visualisation de la mémoire de programme : fenêtre Program Memory.

Dans le menu de **MPLAB**, faire :

View Program Memory

Examiner les informations contenues dans cette fenêtre (soit par l'onglet **Machine**, soit par l'onglet **Symbolic**) : colonne **Address** et colonne **Opcode**.

Expliquer, en utilisant le tableau de la **figure IV.5. page 61** du fichier **Résumé du cours PIC.pdf**, le contenu des cases de mémoire de programme d'adresses **0x0000**, **0x0005** et **0x0006** en faisant notamment le lien avec les instructions assembleur correspondantes qui sont écrites sur la même ligne.

Aide : il s'agit ici de réaliser un assemblage « à la main » comme ce que j'ai fait, sur un exemple bien sûr différent à la **page 62** du fichier **Résumé du cours PIC.pdf**.

Cette fenêtre **Program Memory** permet d'être au plus proche du PIC pendant l'exécution des programmes.

6. Visualisation des variables du programme : fenêtre Watch.

Dans le menu de **MPLAB**, faire :

View Watch

La fenêtre **Watch** de la figure 0.6. s'ouvre alors :

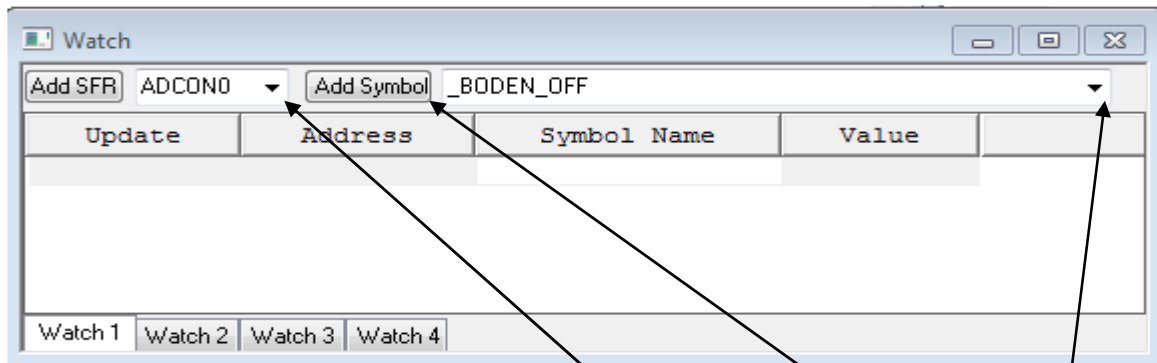


Figure 0.6.

Elle permet de choisir, par son nom symbolique, soit un **SFR**, soit une variable en **GPR**.

Ici, on choisira la variable **tampon** (située en **GPR**) que l'on validera par le bouton **Add Symbol**.

Ce qui donnera le « **Watch** » de la figure 0.7.

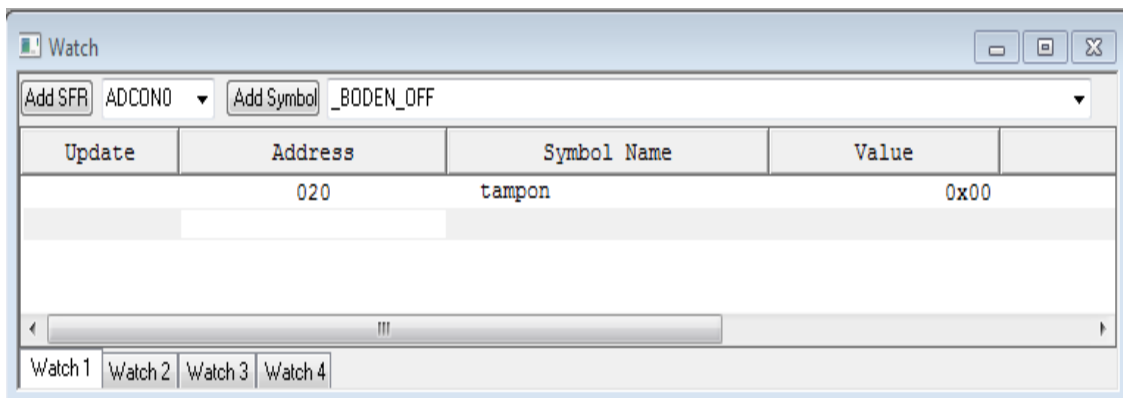


Figure 0.7.

Expliquez, en faisant le lien avec le source, les informations fournies par ce « **Watch** » :

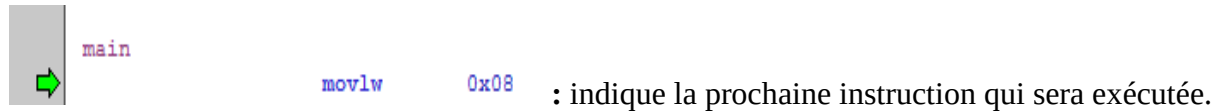
- champ **Address** : pourquoi **0x20** ?
- champ **Value** : pourquoi **0x00** ?

On retiendra : dans un programme, une fenêtre **Watch** permettra de suivre l'évolution d'une variable, ce qui peut être utile pendant les phases de mise au point.

7. Exécution du programme en mode « pas à pas ».

On parle aussi du mode **Single Step** ou de mode **Trace**, tous ces termes étant des synonymes.

Dans ce mode (icône  du simulateur), les instructions du programme sont **exécutées une par une**, avec un arrêt entre chaque instruction, comme indiqué par une petite flèche verte en face de chaque instruction :



Contrairement au mode **Normal** (icône ) où le programme s'exécute sur le simulateur comme s'il fonctionnait sur le PIC.

Le mode « pas à pas » est intéressant pour la mise au point car il permet de voir si le programme effectue bien ce qu'on lui demande de faire, pour peu qu'on fasse du pas à pas « productif », en visualisant entre chaque instruction les registres qui sont concernés (**W**, **SFR** ou **GPR**) et les flags du registre **STATUS** (surtout **Z** et **C**, moins souvent **DC**) qui sont positionnés ou pas.

Pour cela, **en bas de l'écran** de **MPLAB**, sont affichés (cf. figure 0.8.), **après l'exécution** de chaque instruction :

- la valeur du **PC** : le **Program Counter**,
- le contenu de **W** : l'accumulateur de travail (**Working register**),
- les **flags** : en **minuscules** (**z**, **dc** et **c**) s'ils sont à **0** (comme sur la figure 0.8.), en **MAJUSCULES** (**Z**, **DC** et **C**) s'ils sont à **1**.

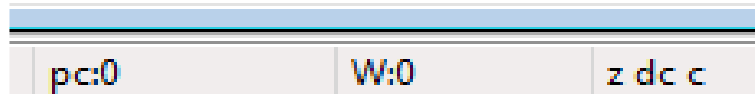


Figure 0.8.

Bien sûr, quand le programme est volumineux, il peut être long et fastidieux de l'exécuter en pas à pas du début à la fin. Il faut « sentir » la zone où se trouve(nt) le (ou les) bugs. Ce sera d'autant plus facile, si vous programmez en utilisant les techniques de la programmation modulaire.

On verra dans le **tutorial N°2** que l'utilisation des « breakpoints » (ou points d'arrêt) peut être également une aide précieuse.

Utilisez l'icône  du simulateur pour exécuter le programme en mode « pas à pas ».

(avec cet icône, on ira aussi **tracer l'intérieur des modules**, contrairement à l'icône  où on exécute le programme principal en pas à pas mais **les modules en mode normal**... ce qui sera intéressant pour sauter les modules trop longs à faire en pas à pas (comme les tempos par exemple) ou les modules qui sont réputés testés à **150 % !!!**))

Regarder l'évolution des registres (**pc** et **W**).

Continuez à tracer jusqu'à la première instruction du module **abs** (et donc jusqu'au label **abs**).

A ce moment, dans le menu de **MPLAB**, faire :

View Hardware Stack

Expliquer la valeur **0x0007** se trouvant au sommet de la pile.

Continuer le mode pas à pas (en examinant et en comprenant bien ce que réalise chaque instruction) jusqu'à l'instruction **nop** se trouvant juste après l'instruction :

call **abs**

Expliquer le contenu de **W** à cet instant. Pour visualiser le contenu de **W** on peut faire :

- soit comme indiqué sur la figure 0.8.
- soit par : **View** **Watch** et **Add SFR** (et en allant chercher **WREG**).

8. Petite modification du module et test avec un paramètre positif, nul puis négatif.

Remplacer les 2 instructions :

andlw **0x80**
btfsc **STATUS,Z**

par une seule instruction venant directement tester le **MSB** de la variable **tampon**.

Reconstruisez le projet.

Faites un reset grâce à l'icône  du menu du simulateur.

Vérifier votre modification en visualisant le résultat dans **W** avec un paramètre **positif, nul** puis **négatif**. Conclusions.

Fin du tutorial N°1 : vous savez maintenant :

- **lancer** l'outil **MPLAB**,
- **ouvrir** un projet existant,
- **visualiser** et **éditer** ses fichiers constitutifs,
- **construire** un projet,
- **choisir** le mode de débogage (ici le simulateur **MPLABSIM**),
- **interpréter** le contenu de la fenêtre **mémoire de programme**,
- **visualiser** des variables,
- **exécuter** un programme en pas à pas,
- **suivre l'évolution** d'un programme en regardant **W** et les **flags**,
- **visualiser** la pile.

III.2. TUTORIAL N°2

Quelques éléments préliminaires théoriques avant de démarrer le tutorial 2 :

Voir pages 13 à 17 du fichier **Résumé du cours pendant TP PIC**.

A la demande des étudiants,
l'enseignant pourra faire une présentation orale de ce tutorial 2.

1. Ouverture et construction du projet tutorial 2.mcp.

Ouvrir le projet **tutorial 2** de **votre** répertoire de travail (dans **d_pic/prog_asm/tutorial 2**)

Construisez le projet **tutorial 2.mcp**.

2. Correction des erreurs de syntaxe et choix de l'outil de débogage.

Corriger les erreurs de syntaxe. En double cliquant gauche sur chaque erreur dans la fenêtre **Output**, on est automatiquement renvoyé sur la ligne du source comportant l'erreur.

Une fois corrigées il vous faut obtenir le : **BUILD SUCCEEDED !!!**

Choisir enfin le simulateur de **MPLAB**, soit **MPLABSIM**.

3. Repérage de la zone des données initialisées et placement d'un breakpoint.

Un élément nouveau est à remarquer dans ce tutorial 2 : c'est la présence de **données initialisées** par les lignes (*écrites ci-dessous en violet*) :

```
extern copy_init_data          ; module copy_init_data dans int_data_bt.asm
I_DATA          IDATA          ; pour la compatibilité avec le fichier script
compteur        DB    0x00
chaine          DB    "Ce tutorial 2 est assez facile",00
```

et par l'appel du module **copy_init_data** au début du main :

```
main
    call    copy_init_data    ; pour initialiser la chaîne de caractères
```

Où se trouve le code de ce module **copy_init_data** dans le **tutorial 2**?

Essayer de tracer votre programme en pas à pas. Que-se-passe t'il ?

Pour éviter de tracer exhaustivement (ce qui serait long et fastidieux !!) le module **copy_init_data** qui est sensé « fonctionner », on va apprendre à utiliser des « **breakpoints** ».

Un **breakpoint** est un point d'arrêt, placé par l'utilisateur à un endroit stratégique et judicieux du programme et où ce programme, s'exécutant en mode **normal** s'arrêtera (s'il y passe bien entendu).

On va donc placer ce « breakpoint » au niveau de l'instruction :

```
        movlw    chaine
```

qui suit l'appel du module **copy_init_data**.

Pour cela, double cliquez gauche sur la ligne assembleur correspondant à l'instruction :

movlw **chaine**

Une petite icône  apparaît sur la gauche de cette ligne, comme indiqué sur la figure 0.9. : ça y est, le breakpoint est implanté !!!

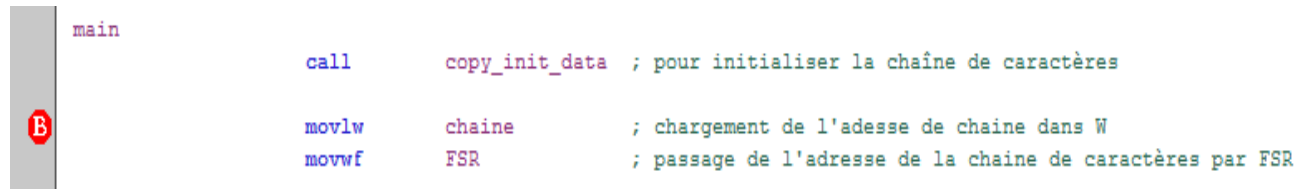


Figure 0.9.

Lancer le programme en mode **normal** (icône  du menu simulateur). Quand s'arrête t'il ?

A bien comprendre : cela signifie que le module **copy_init_data** a bien été effectué (en « temps réel », mais que le programme s'arrête avant d'exécuter l'instruction :

movlw **chaine**

En ouvrant une fenêtre de lecture RAM par : **View** **File Registers**

indiquer l'adresse de la variable **compteur** et sa valeur puis l'adresse du premier caractère et enfin celle du dernier de **chaine**. Ces valeurs d'adresses sont-elles compatibles avec le mapping de la RAM du PIC ? La **chaine** se termine t'elle par l'ASCIIZ ?

Pour vérifier que vous avez bien compris, initialiser la variable **compteur** avec par exemple 0x05 et changer la valeur de **chaine** en tapant une autre chaîne de caractères de votre choix et refaites les manips précédentes (construction, lancement du programme jusqu'au breakpoint et visualisation des variables).

Du coup, expliquez clairement à quoi ont servi les lignes indiquées en **violet** à la page précédente (notamment la directive **DB**).

4. Analyse du main, du module string_length et exécution en pas à pas :

Prenez connaissance du reste du programme (après l'appel du module **copy_init_data**).

Repérer bien les instructions pour le passage du paramètre avant l'appel du module **string_length**. Par quel registre **SFR** est passé ce paramètre ? Est-ce logique et pourquoi ? Commenter pertinemment les lignes concernées.

Analyser le module **string_length** et comprendre son fonctionnement :

- en faisant le lien avec l'**adressage indirect**,
- en faisant le lien avec son équivalent écrit en **langage C**,
- en commentant pertinemment les lignes d'instructions.

Puis exécuter en pas à pas le module **string_length** en contrôlant par une fenêtre **Watch** le contenu du registre **FSR** et de la variable **compteur**. Quelle valeur sera retournée dans **W** ?

Est-ce bien le cas ? Corriger en conséquence le bug du module **string_length**.

5. Test et validation finale :

Placer un seul **breakpoint** à l'endroit le plus efficace de ce programme et tester le module **string_length** avec différentes valeurs de chaîne de votre choix, sans oublier la chaîne **nulle !!!**

**Normalement, lorsqu'un module fonctionne correctement
(et on vient de le vérifier ici en le testant individuellement)
on peut maintenant l'utiliser tel quel au sein d'un autre programme,
(ou le donner à un autre membre de son équipe)
qui aurait besoin, entre autre de déterminer la longueur d'une chaîne de caractères.**

**Si vous avez compris cela et cette méthodologie de travail,
vous commencez à devenir des « as » de la programmation modulaire...**

Fin du tutorial N°2 : vous savez maintenant :

- **visualiser** le contenu de la RAM,
- **définir** des variables **initialisées** en RAM,
- **visualiser** les **SFR** du PIC,
- **corriger des erreurs de syntaxe** d'un programme source,
- **placer** des « **breakpoints** »,
- comment **tester** un module à partir d'un main :
 - passage éventuel de paramètre(s),
 - appel du module par une instruction **CALL**,
 - suivi d'une instruction **NOP** pour placer le breakpoint après l'exécution du module et vérifier si tout c'est passé comme prévu...

A part dans les premiers exercices, où vous apprendrez à vous familiariser avec les principales instructions assembleur, les principales directives d'assemblage, et les différents modes d'adressage, nous essaierons de concevoir autant que faire ce peut nos programmes comme des modules indépendants et donc réutilisables dans un autre contexte.

L'intérêt de cette méthode est loin d'être négligeable. Car avec ces modules de base, on pourra, en les réutilisant (un peu comme des pièces de « lego ») construire des programmes de plus en plus complexes mais qui resteront tout de même « déboguables », contrairement à une écriture « à l'arrache » qui génère un code très dur à mettre au point et à maintenir.

Vous avez bien compris, que dans ces TP, j'accorderai autant d'importance à la méthodologie de travail (structuration de vos programmes et de vos données, rigueur dans la gestion de vos projets...) qu'aux résultats obtenus.

**ON APPELERA L'ENSEIGNANT
POUR FAIRE
UN POINT SUR LES DEUX TUTORIAUX !!!**

Vous êtes maintenant prêts à faire vos propres programmes en assembleur.

BON COURAGE !!!

IV. PREMIERS PAS EN ASSEMBLEUR : A VOUS DE JOUER !!!

IV.1. CREATION DE PROJET ET MASQUAGE

Jusqu'à maintenant, vous n'avez travaillé qu'avec des projets **déjà existants**.

Avant de démarrer cet exercice, on va apprendre comment créer son **propre** projet.

On a vu dans le **tutorial N°1** que la construction d'un projet entraînait la création de nombreux fichiers. Pour éviter d'avoir trop de fichiers dans le même dossier et de risquer de tout « mélanger », je demande à partir de maintenant :

**de ne mettre qu'un seul projet par dossier (répertoire)
portant le même nom que le projet.**

La procédure **obligatoire** (si vous ne la respectez pas, l'enseignant, en plus d'une sanction sur la note finale, ne vous apportera **aucune aide** pendant ces TP) pour créer un nouveau projet et pour respecter cette contrainte est donc la suivante :

Partie « hors » MPLAB :

1. Choisir un nom de projet : prenez un nom « parlant » et « évocateur », rappelant ce que fait votre projet et pas des noms comme « titi », « tata » ou « toto » ou même « exo1 ».

Par exemple dans cet exercice, on appellera le projet **masquage**,

2. Créer un **répertoire** avec le nom de votre projet (ici **masquage**) dans le dossier **prog_asm** de **votre** répertoire de travail,

3. Recopier le **contenu complet** du dossier **vierge_asm** dans le dossier **masquage** que vous venez de créer : il y a 4 fichiers à copier : **vierge_asm** (d'extension **.asm**), **init_data_bt** (également d'extension **.asm**), **P16F877A.INC** et **16f877a.lkr**.

☛ : **attention**, il ne s'agit pas de copier le dossier **vierge_asm** mais bien son contenu !!!

4. Renommer le fichier **vierge_asm** (d'extension **.asm**) en **masquage** (de même extension **.asm**) **donc avec le même nom** que votre projet, comme on l'a imposé plus haut...

Partie « dans » MPLAB :

1. Dans le menu de **MPLAB**, faire :

Project Project Wizard La fenêtre suivante (cf. figure 0.10.) s'ouvre :

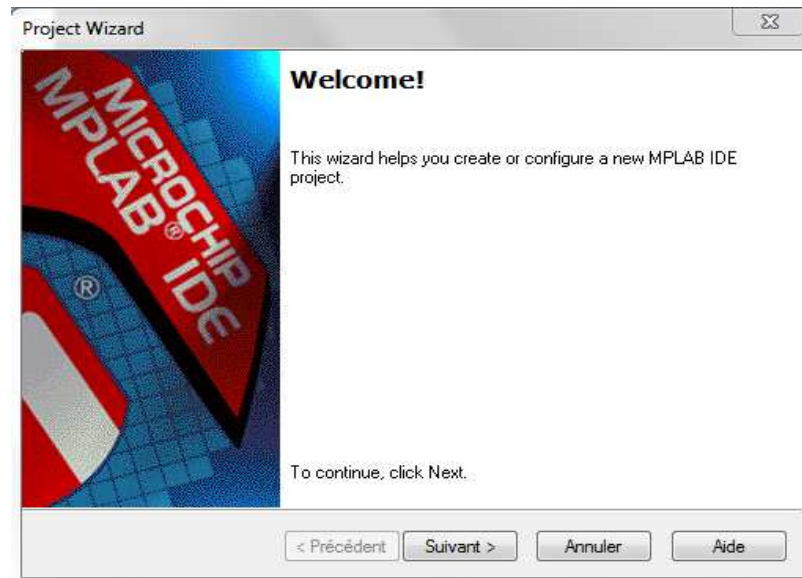


Figure 0.10.

Faire **Suivant**. Et choisir le **PIC16F877A**, comme indiqué sur la figure 0.11.

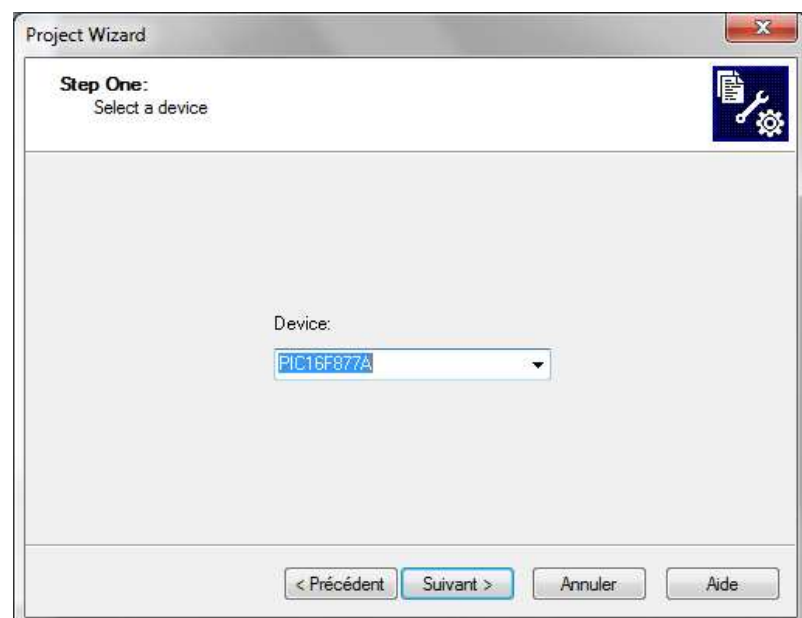


Figure 0.11.

Faire **Suivant**.

Choisir la suite **Microchip MPASM Toolsuite** comme outils de construction de projet, comme indiqué sur la figure 0.12.

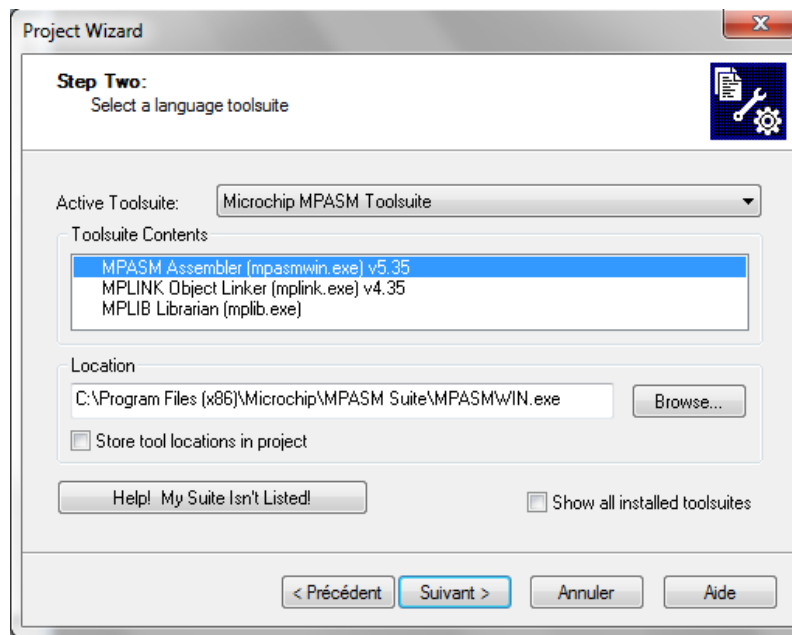


Figure 0.12.

Faire **Suivant**.

Et, par le **Browse**, sélectionner le dossier **masquage** (que vous venez de créer) de votre répertoire de travail et taper **masquage** (et **pas** **masquage.asm** !!!) comme nom de projet. (cf. figure 0.13.)

Puis **Enregistrer** et **Suivant**.

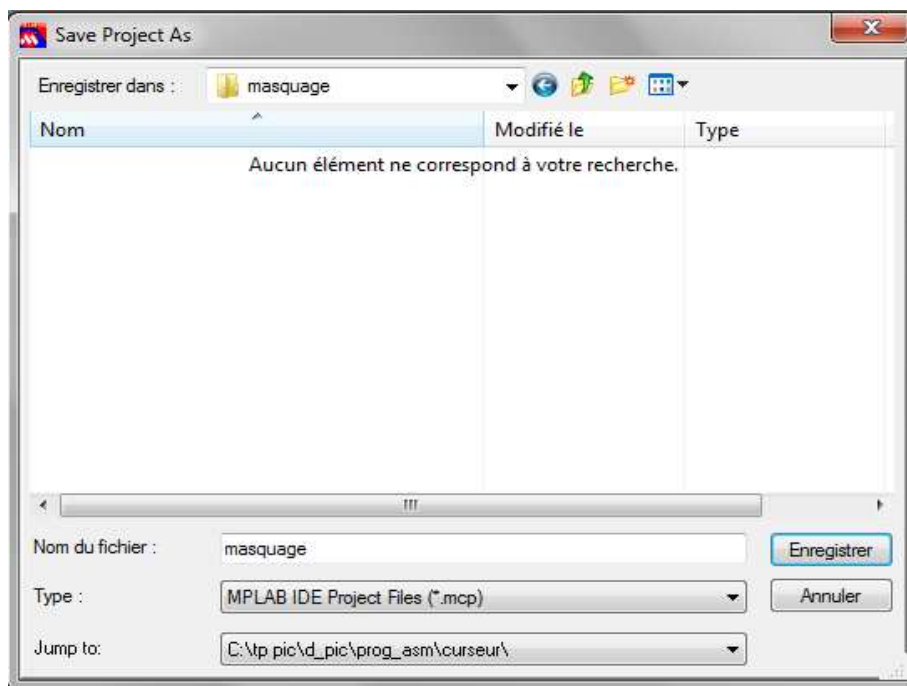


Figure 0.13.

Et enfin, ajouter les4 fichiers **masquage.asm**, **init_data_bt.asm**, **P16F877A.INC** et **16f877a.lkr** à votre projet par **Add** comme indiqué sur la figure 0.14.

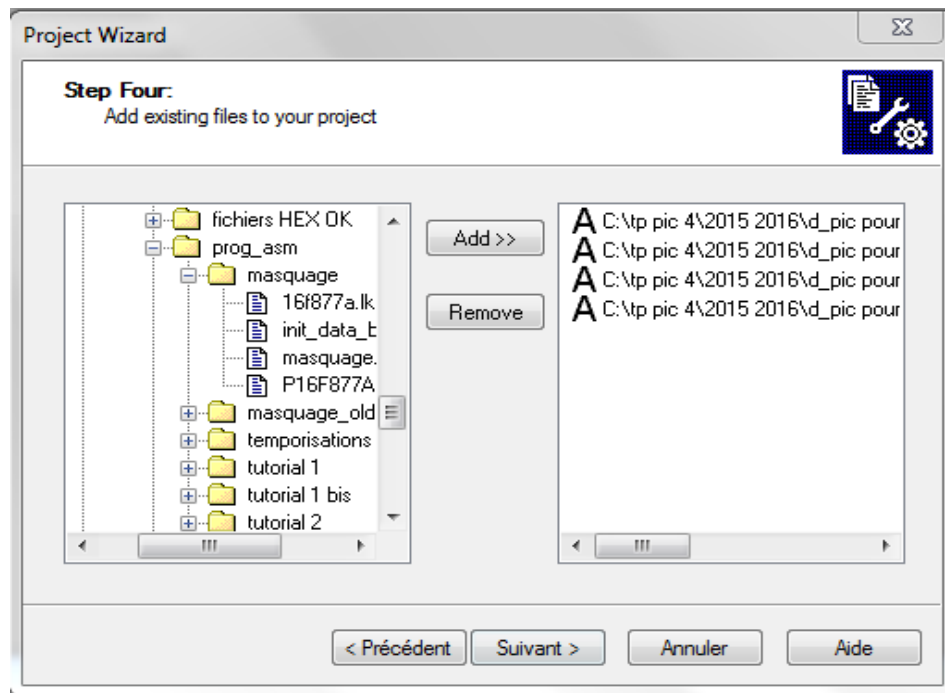


Figure 0.14.

Puis **Suivant**. Et enfin **Terminer**.

Votre projet **masquage.mcp** s'affiche comme le projet courant de travail sous **MPLAB** et comme indiqué sur la figure 0.15 :

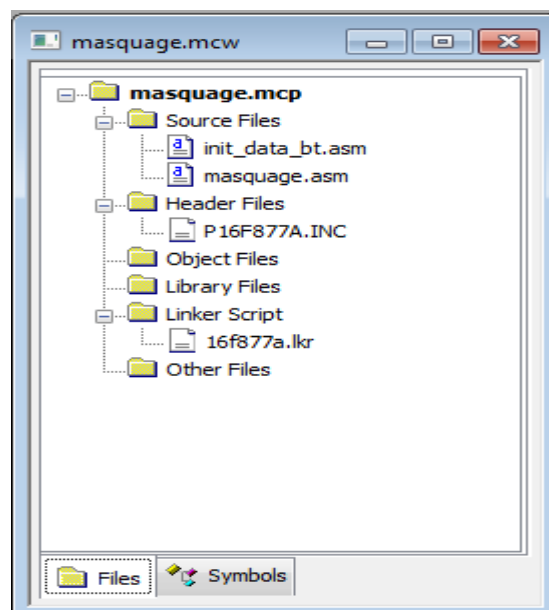


Figure 0.15.

Vous êtes maintenant prêts pour travailler.

Votre base de travail est constituée du fichier **masquage.asm** qui n'est rien d'autre, comme on l'a vu, que le fichier **vierge_asm.asm** et qui constitue une base de départ minimale et structurée pour travailler en assembleur et **surtout qui vous évite de partir de « zéro » à chaque fois !!!**.

En effet, il vous fournit un environnement **correct et suffisant** pour y déclarer vos constantes, vos variables et pour y écrire vos propres instructions.

Vérifier que le projet est **constructible** (même si vous n'avez encore écrit aucune ligne d'assembleur). Si ce n'est pas le cas, inutile d'aller plus loin, c'est que vous avez fait une erreur dans la procédure de création d'un nouveau projet. Cherchez l'erreur avant de continuer à la ligne suivante !!!

Vous n'avez plus ensuite qu'à respecter **le cahier des charges** suivant :

Définir une **variable** ayant pour nom **caractere** et pour taille **un octet**.

Dans votre main, initialiser cette variable avec le code ASCII d'un chiffre de votre choix.

Première façon de faire un masque :

Puis, dans les lignes suivantes de votre main, **forcer à 0 les 4 bits de poids fort** de la variable **caractere** en utilisant la technique de masquage de bits par une instruction simulant un opérateur logique (famille **AND** ou famille **IOR**, à vous de choisir ...) et le masque adéquat. Par exemple, si au départ la variable **caractere** contient **0x35** (code ASCII du chiffre 5) après vos instructions, elle doit contenir **0x05**.

(voir à ce sujet le tableau de la **figure IV.9. page 65 du polycopié Résumé du cours PIC** pour la liste complète des instructions assembleur et le renvoi à la dernière colonne de ce tableau aux pages adéquates (du polycopié de cours complet) pour des informations plus précises sur les instructions et des exemples si nécessaire).

Tester et valider votre programme sous le simulateur.

Proposer une méthode pour **forcer à 1 les 4 bits de poids fort** de la variable **caractere** et tester la comme précédemment.

Deuxième façon de faire un masque :

Mettez en commentaire les lignes du masquage précédent et, à la suite, refaire le **forçage à 0 des 4 bits de poids fort** de la variable **caractere** en utilisant cette fois-ci des instructions bits de la famille des **BxF** (avec x=C ou S).

Tester et valider votre programme sous le simulateur.

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

IV.3. MISE EN PLACE DU BIT DE PARITE

Quelques éléments préliminaires théoriques avant de démarrer cet exercice,
et surtout pour bien comprendre ce qu'on entend par bit de parité !!!

Voir pages 18 à 20 du fichier Résumé du cours pendant TP PIC.

A la demande des étudiants,
l'enseignant pourra faire une présentation orale de ces éléments

Procéder comme précédemment (partie « hors » MPLAB et partie « dans » MPLAB) pour créer un nouveau projet **parite**.

Ecrire en assembleur un module dont le prototype en C est la fonction suivante :

Ce module :

- **reçoit** son paramètre par **W** (taille **8 bits** avec **MSB** = 0)
- **détermine le bit de parité P** (en parité **paire**) de ce paramètre,
- **retourne** dans **W** le paramètre avec son bit de **parité P conforme**.

Aide : penser aux instructions de tests de bits (**btfss** ou **btfsc** associées à des **goto**) pour compter le nombre de bits à 1 dans le paramètre puis pour savoir si ce nombre est pair ou impair.

Tester et valider votre module sous le simulateur en écrivant un main de la forme suivante :

```
movlw      .....      ; mettre dans W le caractère (avec son MSB = 0)

call       parite_paire  ; appel du module que vous avez écrit

nop        ; pour placer un breakpoint après l'exécution de
           ; parite_paire et vérifier le contenu de W
```

Par exemple si en entrée du module **W** contient **0x53** (soit 4 bits à 1), au retour **W** contiendra **0x53** (le bit **P** sera mis à 0).

Mais si en entrée **W** contient **0x43** (soit 3 bits à 1), au retour **W** contiendra **0xC3** (le bit **P** sera mis à 1).

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

Application (à faire de façon optionnelle) si le TP0 est fini dans le « timing » :

Comme cela a été fait dans le tutorial 2, définir une chaîne de caractères en mémoire (directive **DB**). Le but est de positionner le bit de parité de tous les caractères de cette chaîne.

Il faut donc écrire un module **parite_paire_string** qui reçoit en entrée par le registre **FSR** un pointeur sur la chaîne de caractères et qui positionne un à un tous les bits de parité de chaque caractère de cette chaîne. Ce qui revient au prototype C suivant :

void **parite_paire_string** (char *s)

Aide : pour le passage du paramètre par **FSR**, inspirez vous de ce qui a été fait dans le tutorial 2 et utilisez bien sûr le module **parite_paire** précédent !!!

Tester et valider votre module sous le simulateur en écrivant un main qui passe le paramètre par **FSR** et qui appelle le module **parite_paire_string**.

IV.4. NOMBRE D'OCCURRENCES D'UN CARACTERE DANS DANS UNE CHAINE

Procéder **comme précédemment** (partie « hors » **MPLAB** et partie « dans » **MPLAB**) pour créer un **nouveau** projet dénommé **occurrence**.

Comme cela a été fait dans le **tutorial 2**, définir une chaîne de caractères en mémoire (directive **DB**). Le but est de déterminer le nombre de fois qu'une lettre donnée est présente dans cette chaîne de caractères.

Il faut donc écrire un module **occurrence_lettre** qui reçoit en entrée :

- par le registre **FSR** : un pointeur sur la chaîne de caractères,
- par le registre **W** : le code ASCII d'une lettre quelconque.

Ce module retournera dans **W** le nombre d'occurrence de cette lettre.

Ce qui revient au prototype C suivant :

char **occurrence_lettre** (char *s, char c)

Aide : là aussi, pour le passage du paramètre par **FSR**, inspirez vous de ce qui a été fait dans le tutorial 2.

Tester et valider votre module sous le simulateur avec un main adéquat.

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

Un conseil et une obligation de fin de TP N°0 :

Après avoir téléchargé **MPLAB** sur votre PC perso, faites et testez quelques exercices du **chapitre V**. du polycopié de cours complet puis terminez ce TP N°0 chez vous !!!

**Et n'oubliez pas de préparer le TP N°1 pour la prochaine séance.
(cf. page de garde du TP N°1) !!!**

TP N° 1

Objectifs principaux :

Programmation modulaire : modules de temporisations paramétrables
Consolidation des connaissances en assembleur
Gestion de projets « multi sources »
Utilisation du debugger ICDx
Programmation des Entrées/Sorties parallèles
Mise en EEPROM programme du PIC d'une application
Mise en évidence du phénomène de rebonds sur un bouton poussoir
Minuterie à « économie d'énergie »

**Lire et bien comprendre : chapitre VI.1. du polycopié de cours complet :
les temporisations logicielles (sauf VI.1.4. temporisations en C)**

Préparer et écrire tous les modules source du I.2. de ce TP N°1 :

- temporisations fixes : tempo250, tempo1ms, tempo4ms, tempo1s
- temporisations paramétrables : tempo_p_ms, tempo_p_s.

***Si cette préparation n'est pas faite,
aucune chance d'avancer loin dans ce TP N°1 !!!***

N'oubliez pas que je vérifie votre préparation en début de TP !!!

Enseignant : THIEBAUT Bernard

REMARQUES, RAPPELS ET CONSEILS GENERAUX SUR LES TP

Tous les projets et programmes qu'on vous demande de réaliser seront stockés dans votre propre répertoire de travail, qui doit être créé dans le dossier Etudiants/Formation (comme vous l'avez fait dans le **TP N°0**), s'il n'existe pas déjà...

Toute intrusion dans un autre répertoire que le sien pour y stocker des fichiers sera sanctionnée par l'enseignant.

Pensez à sauvegarder tout votre répertoire de travail sur votre support en fin de séance...

Les noms des projets et programmes seront évocateurs et parlants

Un seul projet par dossier portant le même nom que le projet...

Beaucoup d'exercices dans ce TP auraient pu être faits « à l'arrache » plus rapidement mais j'accorde beaucoup d'importance à la méthodologie de travail « structurante » que je vous propose car à mon avis c'est la bonne façon de se donner les bonnes méthodes et habitudes de développement pour faire à terme des projets plus complexes.

I. MODULES DE TEMPORISATION LOGICIELLE

Quelques éléments préliminaires théoriques :

Avant de démarrer cette partie, il est important de comprendre le principe des **temporisations logicielles** (de durée **fixe** et **paramétrable**) réalisées sous forme de module.

Voir pages 21 à 24 du fichier Résumé du cours pendant TP PIC.

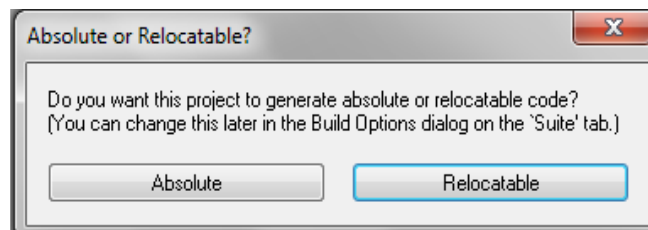
C'est un module de durée fixe que nous allons commencer par tester.

**A la demande des étudiants,
l'enseignant pourra faire une présentation orale de ces éléments.**

I.1. UTILISATION DU « STOPWATCH » DU SIMULATEUR

On va utiliser le projet **existant** **temporisations** que j'ai préparé pour vous.

Ouvrez ce projet sous **MPLAB**. **Il se peut** que vous ayez la fenêtre suivante :



Choisir **Absolute** (génération de code en absolu et non relatif).

Choisir le **simulateur** comme outil Debugger.

Placer un premier **breakpoint** sur la ligne du **call tempo_test**.

Placer ensuite un second **breakpoint** sur la ligne du **nop**.

Construisez votre projet.

Dans le menu de **MPLAB**, faire :

Debugger Settings...

Et dans l'onglet **Osc/Trace**, fixer une fréquence de **4 MHz**, comme indiqué sur la figure I.1. (c'est la fréquence du quartz implanté sur la carte **PICDEM 2 PLUS** que nous utiliserons un peu plus tard dans ce TP). Puis **OK**.

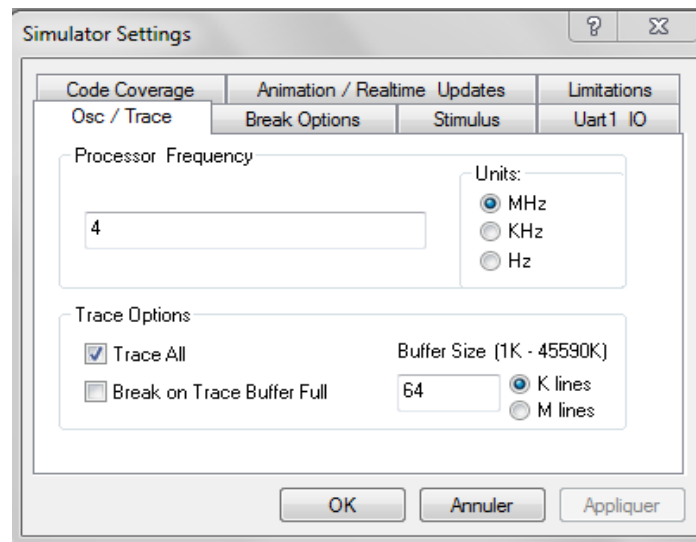


Figure I.1.

Puis, toujours dans le menu de **MPLAB**, faire :

Debugger Stopwatch

pour ouvrir une fenêtre « **Stopwatch** » comme indiqué sur la figure I.2.

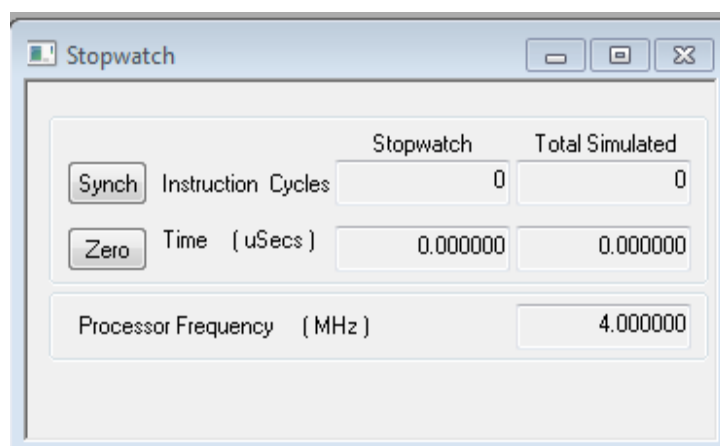


Figure I.2.

Cette fenêtre permet de compter les cycles d'instructions (avec 1 cycle = 1 μ s pour un quartz à 4 MHz comme ici) et par là même des durées d'exécution de bouts de programmes.

Lancer votre programme en mode **normal**.

Au premier **breakpoint** rencontré, remettre à **0** (bouton  de la fenêtre **Stopwatch**) le temps.

Puis relancer à nouveau votre programme en mode **normal**.

Au second **breakpoint** rencontré, vous pouvez lire la durée écoulée entre ces deux **breakpoints**.

A quoi correspond ici cette durée ?

Vérifier la formule théorique prévue et calculée en cours pour la durée **totale** T_1 du module **tempo_test** :

$$T_1 = (3x + 5) \mu s.$$

où x est la valeur **littérale** chargée dans **W** au début de ce module (10 par défaut).

(on pourra rapidement essayer plusieurs autres valeurs).

On dispose donc maintenant de ce module **tempo_test** qu'on va essayer de réutiliser par la technique de la **programmation modulaire**.

L'idée, au final, étant de faire un peu l'équivalent de la fonction **delay** du C ...

I.2. MODULES DE TEMPORISATIONS PARAMETRABLES

Tous les modules suivants à concevoir seront bien sûr et **obligatoirement** écrits dans le projet **temporisations** précédent.

En d'autres termes, ne pas faire un projet par module !!!

En utilisant la méthode par **programmation modulaire** :

- écrire le module **tempo250** d'une durée **fixe** de **250 μ s**, puis tester le sur le simulateur, (on pourra être amené à utiliser l'instruction **nop** pour approcher au mieux 250 μ s)
- écrire le module **tempo1ms** d'une durée **fixe** de **1 ms**, puis tester le sur le simulateur,
- écrire le module **tempo4ms** d'une durée **fixe** de **4 ms**, puis tester le sur le simulateur,
- écrire le module **tempo1s** d'une durée **fixe** de **1 s**, puis tester le sur le simulateur.

Ensuite, et **ensuite seulement**, écrire en assembleur des temporisations **paramétrables** de prototype équivalent (en utilisant bien sûr le travail précédent) :

- void **tempo_p_ms** (unsigned char ms)

L'argument **ms** est nombre de **ms** de la durée souhaitée passée par **W**.

On testera ce module par un main comme indiqué ci-dessous :

```
movlw      .....      ; avec une valeur de test fixe et choisie par vous  
                                     ; penser aussi à tester avec une valeur nulle !!!  
  
call      tempo_p_ms    ; appel du module à tester  
  
nop                                     ; pour mettre un breakpoint au retour du module
```

- void **tempo_p_s** (unsigned char s)

L'argument **s** est nombre de **s** de la durée souhaitée passée par **W**.

On testera ce module par un main comme indiqué ci-dessous :

```
movlw      .....      ; avec une valeur de test fixe et choisie par vous  
                                     ; penser aussi à tester avec une valeur nulle !!!  
  
call      tempo_p_s     ; appel du module à tester  
  
nop                                     ; pour mettre un breakpoint au retour du module
```

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

II. SYNCHRONISATION SUR EVENEMENT EXTERIEUR

Quelques éléments préliminaires théoriques :

Avant de démarrer cette partie, il est conseillé de comprendre le principe des **ports parallèles** du PIC et la façon de les utiliser.

Voir pages 25 à 34 du fichier Résumé du cours pendant TP PIC.

**A la demande des étudiants,
l'enseignant pourra faire une présentation orale de ces points importants sur les PORT.**

C'est la dernière manipulation que nous ferons sous **simulateur**.

Nous passerons ensuite sur la deuxième méthode de travail en TP, en utilisant la carte **PICDEM2 PLUS** en lieu et place du simulateur.

L'évènement extérieur sera ici **l'appui sur un bouton poussoir**.

Nous allons supposer qu'un **Bouton Poussoir (BP)** est relié à la ligne **4** du **PORTA**, comme indiqué sur la figure I.3. ci-dessous.

Que vaut la tension sur la patte **RA4** quand on n'appuie pas sur le **BP** ?

Puis, quand ce même **BP** est enfoncé ?

Tracer l'évolution de la tension **RA4(t)**, en appelant t_A , l'instant d'Appui sur le **BP**.

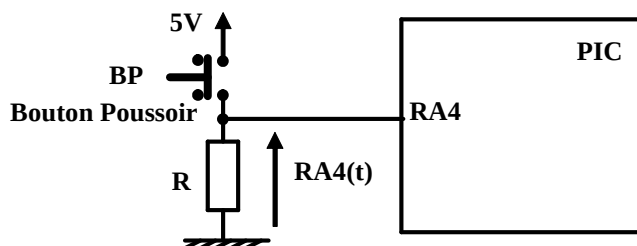


Figure I.3.

Ecrire les trois premières instructions du main de **temporisations.asm**, comme indiqué ci-dessous :

main	btfss	PORTA,4
	goto	main
suite	nop	

En faisant le lien « **hardware/software** » avec le schéma de la figure I.3., expliquer à quoi servent ces trois lignes assembleur.

On va maintenant vérifier votre réponse expérimentalement.

Pour cela, on va simuler l'appui sur le Bouton Poussoir **BP** en envoyant, grâce au simulateur, un « **stimulus** » **asynchrone**.

Dans le menu de **MPLAB**, faire :

Debugger Stimulus New Workbook

La fenêtre « **Stimulus** » s'ouvre alors comme indiqué sur la figure I.4. ci-dessous :

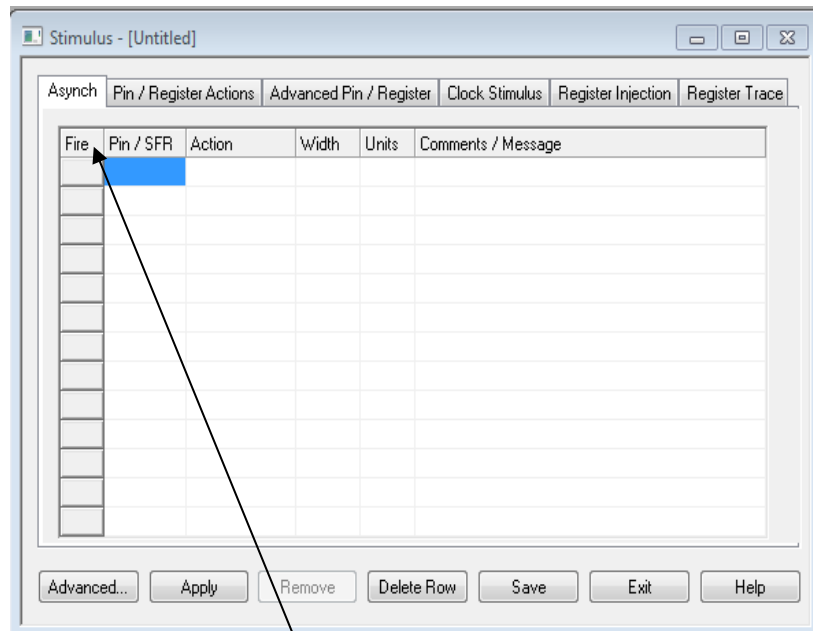


Figure I.4.

Dans l'onglet **Asynch**, choisir **RA4** (colonne **Pin/SFR**), puis l'action (colonne **Action**) **adaptée** simulant l'appui sur **BP** : mettre **RA4** correspondant à **BP non enfoncé**.

A partir de cet instant, l'appui sur le bouton **Fire** de la ligne correspondante enverra le stimulus programmé. Exécuter votre bout de programme en pas à pas. Que se passe t'il ?

Puis simuler **l'appui sur BP** et continuer la simulation en pas à pas.

Conclusions.

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

III. LED ET BP « DANS LE MONDE REEL »

III.1. LECTURE DE SCHEMA

On donne sur la figure I.5. le câblage, sur la carte **PICDEM 2 PLUS** du **BP S2**.

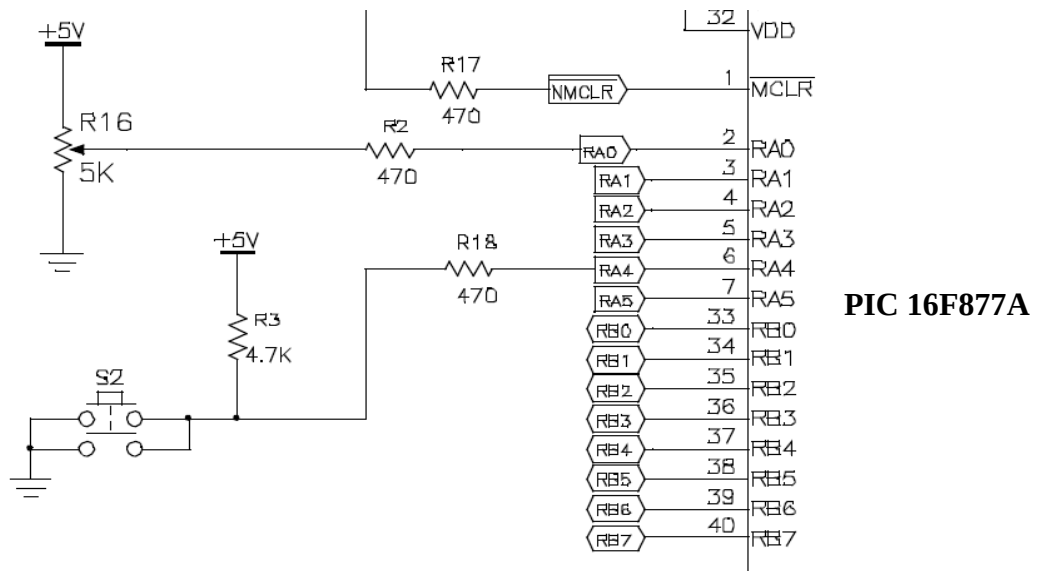


Figure I.5.

Noter attentivement les **différences** par rapport au schéma de la figure I.3. précédemment simulé, notamment au niveau du câblage du bouton poussoir **S2**, branché sur la broche **RA4**.

A votre avis, à quoi sert la résistance R18 de 470 Ω ?

Sur la figure I.6. on indique également comment sont câblées les 4 Leds présentes sur la carte **PICDEM 2 PLUS** et mettant en œuvre 4 lignes du **PORTB**.

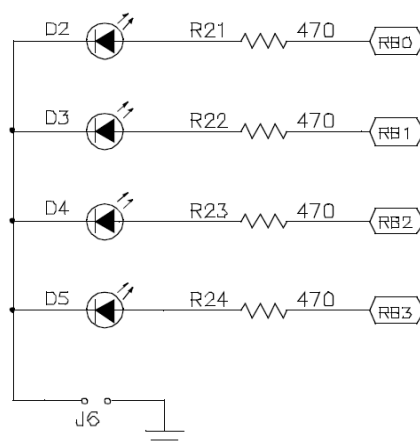


Figure I.6.

A quoi sert le cavalier (ou **Jumper** en anglais) **J6** ?

C'est le moment de passer du monde « virtuel » de la simulation au monde « réel » d'une **vraie carte cible**.

Créer, dans votre répertoire de travail et avec la méthode habituelle, un **nouveau** projet de nom **led et bp** (reportez-vous au **TP 0** si vous avez oublié la méthode...).

Le but dans ce **nouveau** projet est de mettre tout le travail demandé ici, concernant le pilotage des leds et des boutons poussoirs, tout en pouvant exploiter les **temporisations programmables** écrites dans la partie précédente (notamment **tempo_p_ms** et **tempo_p_s**).

A partir de là, il y a **deux façons** de procéder :

A. la méthode « professionnelle » que **je vous conseille** de suivre car elle est plus structurante,

B. la méthode « à l'arrache, quand on est pressé et pas très « propre » dans son travail....
On choisira bien sûr **l'une ou l'autre**...

A. La méthode « professionnelle » :

Elle consiste à structurer au mieux son projet en attachant le fichier **temporisations.asm** au projet **led et bp.mcp**. Mais elle nécessite quelques modifications pour que le nouveau projet **led et bp.mcp**, qui devient alors « **multi-sources** », soit « linkable ». Les voici, mais il est important **de bien les effectuer, sans en oublier aucune**, sinon vous perdrez un temps fou à mettre au point un projet générant un code exécutable exploitable.

① hors **MPLAB**, copier le fichier **temporisations.asm** dans le répertoire **led et bp** de votre nouveau projet,

② dans **MPLAB**, attacher cette version copiée **temporisations.asm** à votre projet **led et bp.mcp** en faisant :

- sélection du répertoire **Sources Files** de votre projet **led et bp.mcp**
- puis cliquer souris droite et **Add Files...**
- aller chercher le fichier **temporisations.asm** dans le répertoire **led et bp**
- et **ouvrir**.

Le fichier **temporisations.asm** doit maintenant apparaître dans le gestionnaire de projets de **led et bp.mcp**.

③ éditer ce fichier **temporisations.asm**,

④ mettre en commentaire la ligne **__CONFIG** : car la configuration est déjà faite dans le fichier **led et bp.asm** et la « multi configuration » est interdite,

⑤ remplacer toutes les lignes suivantes du début du code de **temporisations.asm** :

```
ORG      0x0000
goto     main
ORG      0x0005
main      + le test sur RA4 que vous avez écrit précédemment.
```

par la directive **CODE**

Grâce à cette directive, le fichier **temporisations.asm** sera lié avec le fichier **led et bp.asm** au moment de la construction du projet **led et bp.mcp**.

⑥ tous les modules internes de **temporisations.asm** doivent être déclarés en **global**, pour pouvoir être appelés par les autres sources du projet. Il suffit pour cela de rajouter la ligne suivante dans le fichier **temporisations.asm** (au niveau des fichiers d'inclusion) :

```
global      tempo250,tempo1ms,tempo4ms,tempo1s,tempo_p_ms,tempo_p_s
```

⑦ tous les modules de **temporisations.asm** doivent être déclarés en **extern** dans le fichier **led et bp.asm** pour qu'on puisse les appeler à partir de ce fichier. Il suffit pour cela de rajouter la ligne suivante dans le fichier **led et bp.asm** (au niveau des fichiers d'inclusion) :

```
extern      tempo250,tempo1ms,tempo4ms,tempo1s,tempo_p_ms,tempo_p_s
```

⑧ et enfin attacher (si ce n'est pas déjà fait) le fichier script **16f877a.lkr** (qui doit se trouver dans votre répertoire **led et bp**) à votre projet **led et bp.mcp** en faisant :

- sélection du répertoire **Linker Script** de votre projet **led et bp.mcp**
- puis cliquer souris droite et **Add Files...**
- aller chercher le fichier **16f877a.lkr** dans le répertoire **led et bp**
- et **ouvrir**.

C'est un peu long, mais c'est la bonne méthode pour construire des projets de plus en plus complexes par étapes. Normalement, si tous ces points ont été correctement faits, vous pouvez construire avec succès votre projet **led et bp.mcp**.

Faites le maintenant, (avant même d'avoir écrit une seule ligne nouvelle de programme) et vérifiez que votre projet est bien « constructible ». Sinon, inutile de continuer et chercher l'erreur...

Il ne vous reste plus qu'à continuer (en sautant la partie **B.** suivante) pour savoir ce que je vous demande de faire avec les leds et les boutons poussoirs.

B. La méthode « à l'arrache », plus rapide mais moins structurante :

Dans ce cas là, avec une série de « copiez/collez » rapatriez dans le source de **led et bp.asm** tous les modules de **temporisations.asm** que vous voulez utiliser.

Et dans ce cas là, votre projet **led et bp.mcp** sera « **mono source** » : il ne contiendra que le fichier **led et bp.asm**.

Vous pouvez alors construire votre projet **led et bp.mcp** et, si tout est OK, continuer pour savoir ce que je vous demande de faire avec les leds et les boutons poussoirs.

III.2. CLIGNOTEMENT DE LA LED D2

L'objectif ici est tout d'abord de faire clignoter la led **D2** à un certain rythme.

Bien que ce programme soit relativement simple, nous allons tout de même le concevoir en définissant plusieurs modules (pour rester résolument dans « l'esprit » de la programmation modulaire...)

- écrire un module **init_port** :

qui configure la ligne du **PORTB** concernée, et **elle-seule**.

prototype : void init_port (void) ;

- écrire un module **allume_led_D2** :

qui allume la led **D2**, et **elle-seule**.

prototype : void allume_led_D2 (void) ;

- écrire un module **eteint_led_D2** :

qui éteint la led **D2**, et **elle-seule**.

prototype : void eteint_led_D2 (void) ;

Puis, en utilisant ces 3 modules et le bon module de temporisation, faire un **main** qui fait **clignoter en permanence** la led **D2** suivant le rythme :

- **100 ms** allumée,
- **200 ms** éteinte.

III.2.1. TRAVAIL SOUS ICD3

Il est temps maintenant de tester ce programme sur la carte cible **PICDEM 2 PLUS**.

Reliez le boîtier **ICD3** et la carte cible **PICDEM 2 PLUS** comme indiqué sur la [page 3](#) du fichier [Résumé du cours pendant TP PIC](#). Ne pas oublier également l'alimentation de la carte cible.

Puis, dans le menu de **MPLAB**, faire :

Debugger Select Tool MPLABICD3

pour indiquer à **MPLAB** que vous « débutez » avec le module **ICD3**.

Il faut ensuite construire votre projet **led et bp** pour être compatible avec cet outil.

Et, s'il n'y a pas bien sûr d'erreurs de construction, pour envoyer votre exécutable vers le boîtier ICD3 et la carte cible, il faut faire :

Debugger Program

Ce qui entraînera (normalement) le message suivant dans la fenêtre **Output** :

Programming...
Programming/Verify complete

On peut alors lancer le programme en **mode normal** et « **admirer** » son travail sur la carte cible !!!

Remarque : on peut aussi sous **ICD3** faire du pas à pas, des breakpoints ... bref tout ce qu'on faisait sous le simulateur, mais là votre programme tourne dans son environnement réel ce qui n'est pas tout à fait la même chose : une étape a été franchie vers la réalisation concrète de l'application.

III.2.2. PROGRAMMATION DU PIC

Vous vous dites sûrement : ce serait bien que cette petite application puisse s'exécuter directement sur la carte cible chaque fois qu'on la met sous tension, ce qui la rendrait totalement autonome (et on n'aurait plus besoin ni du **PC**, ni du boîtier **ICD3** pour la faire fonctionner).

C'est possible, et c'est la troisième façon de travailler avec la carte cible.
(cf. [page 4](#) du fichier [Résumé du cours pendant TP PIC](#)).

Dans le menu de **MPLAB**, faire :

Programmer **Select Programmer** **MPLABICD3**

pour indiquer à **MPLAB** que vous utiliserez le module **ICD3** pour programmer l'EEPROM de la mémoire de code du PIC de la carte **PICDEM 2 PLUS**.

Faire ensuite :

Programmer **Program.**

(une reconstruction du projet peut vous être demandée à ce moment là : acceptez-la !!!)

C'est parti : le fichier exécutable de votre projet est mis dans l' EEPROM programme du PIC.

Ce qui entraînera (normalement) le message suivant dans la fenêtre **Output** :

Programming...
Programming/Verify complete

Déconnecter la carte cible du module **ICD3**.

Faites un reset en appuyant sur le bouton poussoir **S1** (démarrage à chaud) ou enlever et remettre l'alimentation de la carte cible : c'est votre programme qui s'exécute (il a pris le contrôle total de la carte **PICDEM 2 PLUS**).

Remarque importante :

C'est par ce qu'on a géré le **vecteur de Reset**, grâce à l'instruction :

goto main

située à l'adresse **0x0000**,

que le PIC peut se brancher sur ce programme applicatif.

Sans cela, le système aurait purement et simplement planté !!!!

Ce n'est pas très spectaculaire mais **imaginez** maintenant de faire (on ne vous demande pas de le faire réellement, mais bien d'imaginer...) :

- un module void **tempo1mn** (void) : tempo **fixe** de 1 minute (facile avec **tempo1s**),
- un module void **tempo1h** (void) : tempo **fixe** de 1 heure (facile avec **tempo1mn**),
- un module void **tempo_p_mn** (char mn) : tempo variable de mn minutes (facile avec **tempo1mn**),
- un module void **tempo_p_h** (char h) : tempo variable de h heures (facile avec **tempo1h**),
- remplacer la led **D2** par un relais mettant en route votre cafetière : vous serez réveillés demain matin par une douce odeur de café. C'est déjà plus intéressant !!!

Pas convaincus : connectez un écran tactile sur la carte **PICDEM 2 PLUS** et vous aurez votre ipho...

Mais là, je ne m'engage pas pour demain car il y aura un peu plus de développement logiciel !!!

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

III.3. BOUTON POUSSOIR S2 ET LED D2

L'objectif ici est de réaliser une **petite minuterie** de base avec le **BP S2** et la led **D2**.

En gardant le même projet (« commentez » par des « ; » votre travail précédent) faire un main qui allume la led **D2** pendant 5 secondes à chaque appui sur **S2**. (principe d'un monostable « non retrigrable »).

(le test pourra se faire au choix : soit sous **ICD3**, soit directement dans l'EEPROM du PIC...).

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

III.4. COMPTEUR D'APPUIS SUR BOUTON POUSSOIR S2

Comme on le voit sur le schéma de la figure I.6., la carte **PICDEM 2 PLUS** possède 4 leds qu'on peut utiliser pour afficher en binaire la valeur d'un quartet. (**D5** en poids **fort** et **D2** en poids **faible**).

Toujours en gardant le même projet (« commentez » par des « ; » votre travail précédent) faire un main qui compte le nombre d'appuis sur **S2** et affiche le résultat du comptage sur les 4 leds **D5** à **D2**.

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en assembleur**
 le fichier : **compte appui S2.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

Aide : un module intéressant à faire pour cette application est le prototype suivant :

void affiche (char nombre)

qui affiche les 4 bits de poids faibles du paramètre nombre sur les 4 leds.

(**exemple** : si nombre = 0x05 = B'0000 0101' alors : **D5** et **D3** éteintes, **D4** et **D2** allumées)

Le passage du paramètre nombre pourra être fait par **W**.

Tester **individuellement** ce module **affiche** (avant de l'utiliser pour compter le nombre d'appuis sur **S2**). Par exemple en faisant un main qui compte en permanence en envoyant le résultat de ce comptage vers les 4 leds grâce au module **affiche**.

Remarque :

Il se peut que la led **D5**, relié à la ligne **RB3** du **PORTB** ne s'allume pas.

Regarder la configuration choisie par la directive **__CONFIG** et replonger vous dans le tableau de la **figure III.18.** du polycopié de cours complet. Faites le lien avec le bit **RB3** et corriger la configuration en conséquence. La led **D5** devrait maintenant s'allumer.

Lorsque le module **affiche** fonctionnera correctement (**et pas avant**), vous pouvez l'utiliser pour compter les appuis sur le **BP S2**.

Normalement, votre programme ne devrait pas marcher si un certain nombre de précautions ne sont pas prises.

La plus importante est liée au phénomène des **rebonds** sur un **BP**.

Qu'est ce que c'est ?

Tout contact mécanique n'est pas **idéal** : quand on appuie sur un **BP**, le court-circuit se fait, puis le contact est rompu, puis remis, puis rompu... un certain nombre de fois suivant la qualité du **BP**.

C'est ce qu'on appelle des **rebonds**.

Quelles sont les conséquences sur la tension présente sur l'entrée **RA4** ?

Elles sont indiquées sur la figure I.7. suivante, où est représentée l'allure de la tension $V_4(t)$ vue par la broche d'entrée **RA4** du PIC :

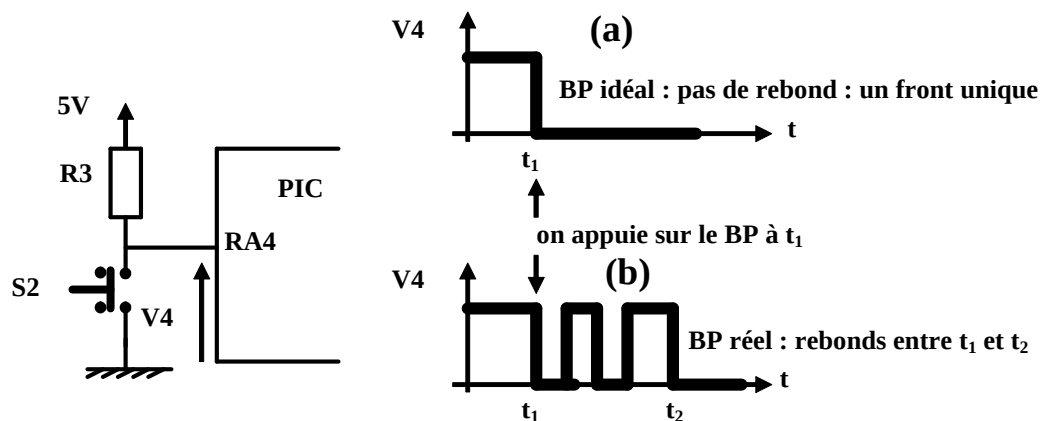


Figure I.7.

(a) BP **idéal** (n'existe pas !!) : pas de rebonds

(b) BP **réel** : présence de rebonds

Le nombre de rebonds est parfaitement aléatoire et la durée $t_2 - t_1$ du phénomène est de l'ordre de la centaine de ms.

Du coup, si votre programme va bien détecter le premier front descendant de $V_4(t)$, compte tenue de la durée des rebonds (de l'ordre de la **centaine de ms**) qui est très grande devant celle d'exécution de votre programme (de l'ordre de la **dizaine de μs**), il va aussi détecter les rebonds et les interpréter comme des appuis successifs.

Ce qui n'est pas **admissible**, puisqu'il n'y a eu, dans ce cas, qu'un seul appui.

Pour éviter ce problème, plusieurs solutions sont possibles, qu'on appelle, dans le jargon technique, des systèmes « **anti-rebonds** ».

- solution « hardware » :

Comme ce qui est fait pour le bouton poussoir **BP S3** relié à l'entrée **RB0** et indiquée sur la figure I.8. ci-dessous :

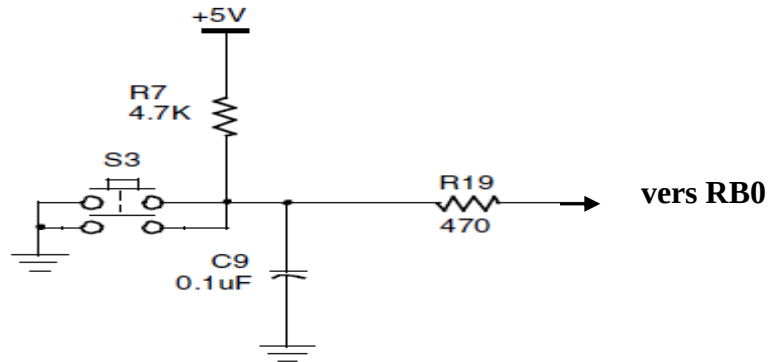


Figure I.8.

La capacité C9 et la résistance R7 filtrent les rebonds qui ne sont plus vus par l'entrée **RB0**.

- solution « software » :

A vous de la trouver, sachant que la durée des rebonds est de l'ordre de la centaine de ms ...

Tester votre solution et valider la.

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

IV. MINUTERIE A « ECONOMIE D'ENERGIE »

La petite minuterie faite dans la **partie III.3.** n'est pas très économique : en effet, à chaque appui sur **S2**, la lumière (ici la led **D2**) reste allumée pendant la durée programmée, même si on n'en a plus besoin.

On va donc rajouter la contrainte suivante :

si on **re-appuie** sur **S2** quand **D2** est **allumée** alors on **éteint** la led **D2**.

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en assembleur**
le fichier : **minuterie economique.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

Nous pourrions d'ailleurs voir sur cet exemple l'intérêt du principe des **interruptions matérielles** (que nous ne mettrons cependant pas en œuvre ici, mais qui sera vu dans le TP suivant).

Je vous propose, sur la figure I.9. un **organigramme possible** pour réaliser cette application.

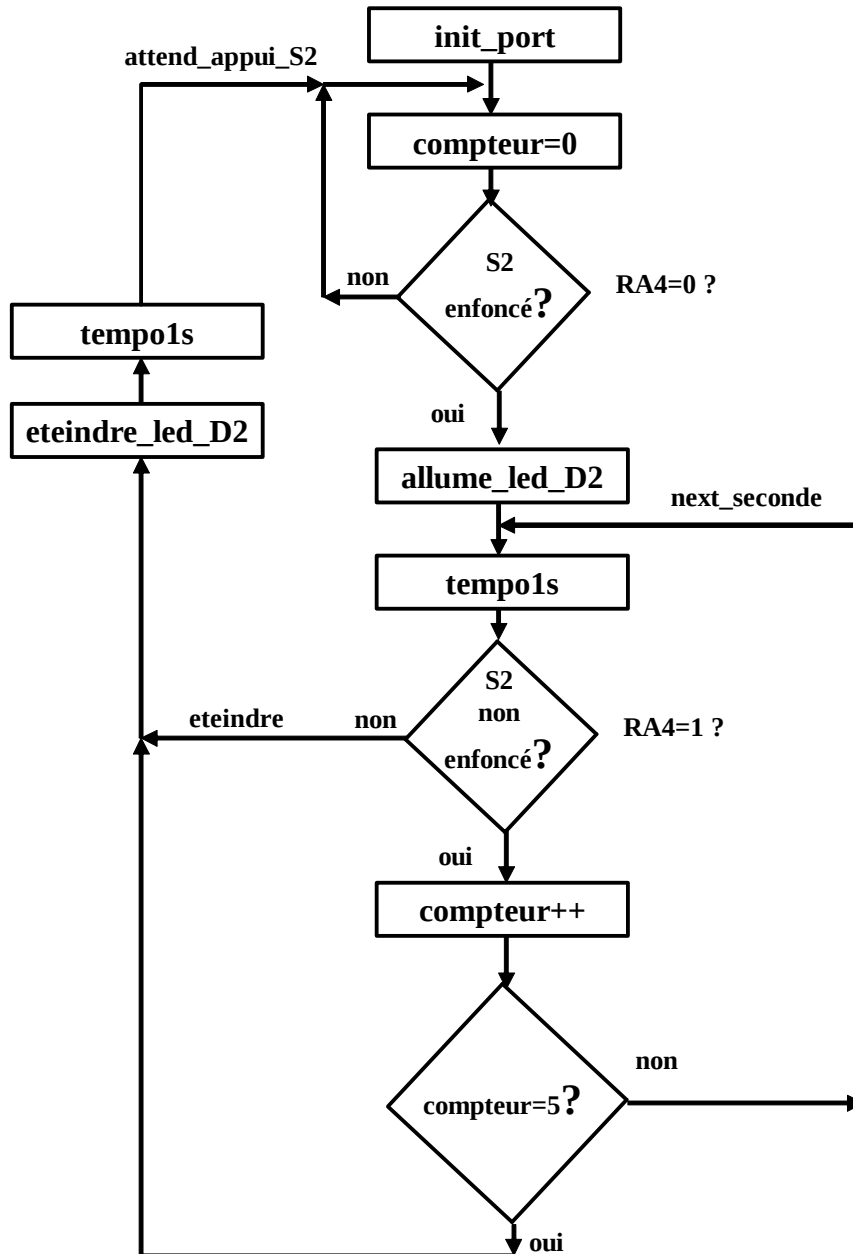


Figure I.9.

Les **labels** `attend_appui_S2`, `eteindre` et `next_seconde` ont été rajoutés sur cet organigramme pour faciliter l'écriture de votre source en assembleur (accès par une instruction **goto**).

Il est conseillé de bien le comprendre et de le respecter pour se donner toutes les chances d'arriver au résultat souhaité.

Remarque :

On voit sur cet organigramme qu'on n'a pas lancé une tempo de 5 secondes en un **seul coup**, mais en **plusieurs bouts**. Pourquoi ?

Répondre à cette question, c'est se donner une clef pour comprendre l'intérêt des interruptions matérielles....

Pourquoi faut-il appuyer plus longtemps sur **S2** pour éteindre **D2** que pour l'allumer ?

Proposer une solution pour remédier à ce problème et tester la.

Quand votre application fonctionnera sous **ICD3**, mettez la en EEPROM.
(méthode décrite plus haut dans la partie **III.2.2.** de ce **TP N°1**).

Et faites la vérification finale sur la carte (autonome) c'est-à-dire déconnectée de ses outils de développement.

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

Fin de TP N°1 :

**Et n'oubliez pas de préparer le TP N°2 pour la prochaine séance.
(cf. page de garde du TP N°2) !!!**

TP N° 2

Objectifs principaux :

Prise en main du langage C
Programmation des Entrées/Sorties logiques
Génération de son par commande d'un buzzer piézoélectrique
Les interruptions matérielles
Mise en œuvre des EEPROM internes
Utilisation du CAN

Sur le polycopié de cours complet :

Lire chapitre VIII.1.1. Généralités et principes des PORTS TOR

Lire chapitre VIII.2. Première approche des interruptions matérielles

Sur le polycopié du TP N°2 :

Lire partie I de ce TP N°2: le TUTORIAL en C

Préparer :

Le programme minuterie.c de la partie II. de ce TP N°2

Les deux fonctions buzzer_on et buzzer_off de la partie III. de ce TP N°2

*Là aussi, si cette préparation n'est pas faite,
aucune chance d'avancer loin dans ce TP N°2 !!!*

N'oubliez pas que je vérifie votre préparation en début de TP !!!

Enseignant : THIEBAUT Bernard

Tous les conseils de programmation fournis dans les précédents TP restent bien sûr à l'ordre du jour.

Nous allons dans ce TP et le suivant privilégier la programmation en langage C.

Rappeler les **avantages/inconvénients** de la programmation en C vs la programmation en assembleur (quelques éléments sont fournis sur la page **189** du polycopié de cours complet).

La prise en main sera faite de la même façon que pour le langage assembleur grâce à un petit tutorial.

Mais les étapes y seront **moins détaillées** que précédemment car beaucoup de choses ont déjà été vues.

Vous pourrez toujours vous reporter au texte du **TP N°0** en cas de petit oubli de votre part.

On vous rappelle qu'en langage **C ANSI** de base, on disposera des bibliothèques comme :

- les fonctions d' IO sur la COM de la carte cible :	stdio.h	(printf, getchar ...)
- les fonctions de gestion des caractères :	ctype.h	(isalpha, toupper ..)
- les fonctions de gestion des chaînes de caractères :	string.h	(strlen, strcmp ...)
- les fonctions standards et d'allocation mémoire :	stdlib.h	(atoi, malloc ...)
- les fonctions mathématiques :	math.h	(sqrt, pow ...) ...

Et bien d'autres encore... comme vous pourrez le voir et les trouver dans la documentation du C fournie sur les PC des labos.

I. « TUTORIAL » EN C

Dans ce tutorial, nous commencerons par la méthode **par simulation** puis nous passerons au **débugage sous ICD3** avant d'éventuellement **programmer** l'application sur l'EEPROM de code du PIC de la carte cible.

Mais vous pouvez dès à présent, télécharger **la suite de développement HI-TECH Universal ToolSuite PICC** qui est en principe disponible sur leur site ou celui de **MICROCHIP** mais en version limitée dans le temps. Cela devrait tout de même vous permettre de pour pouvoir travailler avec votre PC personnel et de progresser en autonomie chez vous.

Quelques éléments préliminaires théoriques :

Avant de démarrer ce **tutorial**, il est important de voir les différentes façons de réaliser des **temporisations logicielles** en C et les différentes manières de **positionner** les bits individuels en C. **En pratique on utilisera la technique par nom symbolique.**

Voir pages 35 à 40 du fichier Résumé du cours pendant TP PIC.

**A la demande des étudiants,
l'enseignant pourra faire un rapide résumé des notions importantes.**

1. Lancement de l'outil de développement

2. Ouverture du tutorial

Ouvrir le projet **tutorial** de **votre** répertoire de travail, présent dans le sous-dossier **d_pic/prog_c/tutorial**.

Prenez connaissance du fichier **tutorial.c**.

Notez que certaines des fonctions écrites ici en C ne vous sont pas totalement inconnues car vous les avez déjà écrites vous-même en assembleur.

Que fait ce programme ?

Rajoutez les commentaires aux endroits indiqués.

De plus, il y a une petite erreur dans le module **init_port**. Corrigez la d'abord avec la technique des masques, puis, pour une meilleure lisibilité, avec l'utilisation du nom symbolique adéquat. Dans la suite du TP, on utilisera le plus souvent possible la technique des noms symboliques. **Ce sera une obligation pour vous !!!**

Aide : RB0 est une sortie ...

3. Construisez votre projet

Si tout se passe bien (et ça doit être le cas ici) vous devrez obtenir dans la fenêtre **Output** :

***** **Build successful!** *****

Quand vous construirez vos propres projets toutes les erreurs et « warnings » du C seront reportées dans cette fenêtre.

En double cliquant **gauche** sur ces erreurs, vous serez directement reliés à la ligne du source en erreur.

4. Lien avec l'assembleur (**facultatif** : vous pouvez sauter cette partie 4.)

Beaucoup de fichiers ont été fabriqués par la construction de votre projet.

Parmi ceux-ci, éditez le fichier **tutorial.lst**.

Puis retrouvez dans ce fichier et **expliquez** les lignes assembleur codant l'instruction C suivante :

RB0 = 1 ;

Aide : on pourra remplacer dans le source RB0 par RB1 (ou RB2...) et regarder les conséquences sur le fichier .lst.

5. Choix de l'outil de débogage

On choisira dans un premier temps le **simulateur**.

Ecrire en C la fonction void **tempo1s** (void) qui effectue une temporisation **fixe de 1 seconde**. (commencez par écrire une fonction void **tempo1ms** (void) d'une durée fixe de **1 ms** et utilisez la pour écrire ensuite la fonction void **tempo1s** (void))

Etalonnez ces fonctions en utilisant la fenêtre **StopWatch** du simulateur. (vous pouvez vous reporter à la partie **I.1.** du **TP N°1** pour l'utilisation de cette fenêtre pour mesurer, sous simulateur, des intervalles de temps entre deux **breakpoints...**).

Remarque :

*Notez que comme en assembleur, vous avez la possibilité de mettre des **breakpoints**, de **tracer** vos programmes, de les lancer en mode normal, de visualiser vos variables par des fenêtres **Watch** ... toutes ces aides étant précieuses pendant les phases de mise au point.*

6. Vérification de votre programme sous ICD3 puis dans l'EEPROM de la carte cible.

Là aussi, admirez le résultat.

Conclusions : vous êtes plutôt assembleur ou plutôt C ? Expliquez pourquoi.

7. Application :

En utilisant comme base ce **tutorial**, réalisez un petit « chenillard » avec les 4 leds **D2 à D5**. Mais pour éviter d'avoir à écrire 8 fonctions différentes pour commander les 4 leds, vous réaliserez une **seule** fonction **générique** du type :

void **led** (unsigned char **led**, unsigned char **action**)

Le premier argument **led** correspond à la led concernée (**D2 à D5**) et le second **action** représente l'action à réaliser sur cette led (l'allumer ou l'éteindre). Il faudra auparavant définir ces différents éléments par une série de **#define** comme par exemple (et à **compléter**) :

```
#define    D2    ...    // identifiant pour la led D2
#define    D3    ...    // identifiant pour la led D3
#define    D4    ...    // identifiant pour la led D4
#define    D5    ...    // identifiant pour la led D5

#define    on     ...    // pour allumer
#define    off    ...    // pour éteindre
```

Par exemple, l'appel suivant dans le main : led (D3,off) éteindra la led **D3**.

Penser aussi à jeter un œil sur les autres conseils de la page suivante...

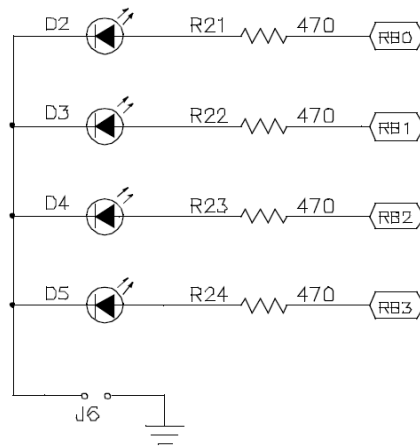
(pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
le fichier : **chenillard.hex** (c'est un exécutable)

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...).

Quelques autres conseils pour réaliser ce petit chenillard :

- penser à initialiser correctement le **PORTB** (avec les noms symboliques adéquats),
(on se reportera à la figure ci-dessous : **toujours un œil sur le hardware !!!**)



- la durée d'allumage de chaque led sera d'une centaine de ms : on pourra en conséquence écrire une fonction de temporisation **paramétrable** de prototype :

```
void tempo_p_ms (unsigned char ms) // ms : durée de la temporisation
```

- on pourra être amenés à déclarer une variable globale **debug** situé à l'adresse **0x70** par :

```
char debug @0x70
```

pour éviter un conflit entre vos propres variables et le débogueur **ICD3** (qui utilise l'adresse **0x70** pour son propre fonctionnement),

- dernier point : pour allumer la led **D5** et/ou inhiber le **Watchdog**, vous pouvez être amenés à décommenter la ligne de configuration suivante :

```
// __CONFIG(HS & WDTDIS & BOREN & LVPDIS );
```

ON APPELERA L'ENSEIGNANT POUR FAIRE UN POINT SUR LE TUTORIAL EN C !!!
--

Vous êtes maintenant prêts à faire vos propres projets/programmes en C.

BON COURAGE !!!

II. MINUTERIE « A ECONOMIE D'ENERGIE » EN C

Quelques éléments préliminaires théoriques :

Avant de démarrer ce nouvel exercice, il est important de voir comment tester les bits individuels en C.

Voir pages 41 à 43 du fichier Résumé du cours pendant TP PIC.

**A la demande des étudiants,
l'enseignant pourra faire un rapide résumé des notions importantes.**

Comme en assembleur, on ne mettra **qu'un seul projet par dossier.**
(car au final, c'est plus facile de s'y retrouver !!!). Il faut donc maintenant créer le projet minuterie.mcp.

La démarche à suivre pour respecter cette contrainte est tout à fait analogue à celle utilisée pour les projets écrits en assembleur. Nous ne la détaillerons donc pas autant...

Hors MPLAB :

1. Créer un répertoire avec le nom de votre projet (ici **minuterie**) dans le dossier prog_c de votre répertoire de travail,
2. Recopier le contenu complet du dossier **vierge_c** dans le dossier **minuterie** que vous venez de créer (il y a 3 fichiers à copier : **vierge_c** (d'extension .c), **pic.h** et **pic168xa.h**),
3. Renommer le fichier **vierge_c** (d'extension .c) en **minuterie** (de même extension .c) **donc avec le même nom** que votre projet, comme on l'a imposé plus haut...

Dans MPLAB :

Faire : **Project** **Project Wizard** Et laisser vous guider :

Choisir le **PIC16F877A** puis faire **Suivant**.

Choisir la suite **HI-TECH Universal ToolSuite** comme outils de construction de projet.

Faire **Suivant**.

Et, par le **Browse**, sélectionner le dossier **minuterie** (que vous venez de créer) de votre répertoire de travail et taper **minuterie** (et **pas** minuterie.c !!!) comme nom de projet.

Puis **Enregistrer** et **Suivant**.

Et enfin, ajouter les fichiers **minuterie.c**, **pic.h** et **pic168xa.h** à votre projet par **Add**.

Puis **Suivant**. Et enfin **Terminer**.

Votre projet **minuterie.mcp** s'affiche comme le projet courant de travail sous **MPLAB**.

Reprenez l'exercice IV. du TP N°1 précédent en utilisant le fichier **minuterie.c**.

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
 le fichier : **minuterie economique.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

Repérer sur l'organigramme fourni sur la **figure I.9. du TP N°1** les structures de haut niveau du type « boucle while » par exemple et écrire cette petite application en langage C.

Pour la mise au point de votre programme, travaillez sous **ICD3**.

Remarque :

Là aussi, on peut être amenés à inhiber le **Watchdog** du PIC.

Pour cela deux méthodes sont possibles :

- ① sous **MPLAB** : il faut aller dans le menu **Configure** de **MPLAB** puis ouvrir la fenêtre **Configuration Bits...** et mettre **WDT** à **off**. La nouvelle configuration inhibant le **Watchdog** sera prise en compte au nouveau chargement sur la carte cible.
- ② ou **mieux** on peut décommenter la ligne **__CONFIG(...)** ; du C.

Puis, quand tout fonctionnera correctement, vous pourrez implanter directement votre programme sur la carte **PICDEM 2 PLUS**.

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

**Pour tous les exercices qu'on vous demandera de faire
dans la suite de ce TP (mais aussi dans le TP N°3),
on vous demande de réaliser un dossier par projet,
en utilisant la méthode vue dans l'exercice précédent
et que vous devez maintenant bien maîtriser !!!
Si cette méthode n'est pas respectée,
vous mélangerez tous vos projets,
et perdrez un temps fou pour vous y retrouver...
Et en plus, aucune aide de ma part !!!**

III. INTRODUCTION AUX INTERRUPTIONS MATERIELLES

Avant de démarrer cette partie, il est important d'avoir pris connaissance des points importants permettant une **première approche sur les interruptions matérielles**.

Voir pages 44 à 56 du fichier Résumé du cours pendant TP PIC.

**A la demande des étudiants,
l'enseignant pourra faire une présentation orale de cette partie.**

On va, dans cette partie, implanter une interruption matérielle. Cette interruption sera exécutée à chaque appui sur le bouton poussoir **S3** (relié à la ligne **RB0** du **PORTB** qui est alors programmée comme une **entrée** logique).

Remarque : en fonction de vos compétences en électronique analogique, vous pourrez admettre **qu'il faut enlever le Jumper J6** pour avoir un front suffisant **5V-0V** pour générer une interruption sur la ligne **RB0**. Du coup, enlevez **J6** et passez directement à la question 5.

1. Prenez connaissance de la façon dont le bouton poussoir **S3** est câblé en regardant le schéma de la carte **PICDEM 2 PLUS** dans le répertoire **documentations...** ou reportez vous au schéma de la **figure I.8.** du **TP N°1**.

Prenez en compte **aussi** le fait que la led **D2** est aussi branchée sur la ligne **RB0** (cf. **figure I.6.** du **TP N°1**).

2. Compte tenu des valeurs des résistances **R7**, **R19** et **R21** en déduire la tension présente sur la borne **RB0** quand le **BP S3** n'est pas enfoncé (on pourra évaluer, dans un premier temps, le courant circulant dans **D2**). Faites le lien avec la luminosité de la diode **D2** dans ce cas.

3. Même question lorsque le **BP S3** est enfoncé. Faites le lien avec la luminosité de la diode **D2** dans ce cas.

4. En déduire que pour envoyer un **front descendant** entre **5V** et **0V** sur la ligne **RB0**, il faut enlever le Jumper (cavalier) **J6** situé à côté des diodes. **Faites le.**
(si **J6** n'est pas enlevé, on a un front entre **2V** et **0V**, ce qui n'est pas suffisant)

5. Pour pouvoir avoir une action « visible » (on devrait ici plutôt dire « audible » car les leds ont été inhibées) lorsqu'on appuie sur **S3**, on va utiliser le buzzer **PIEZO_BUZ P1** présent sur la carte, comme rappelé sur la figure II.1.

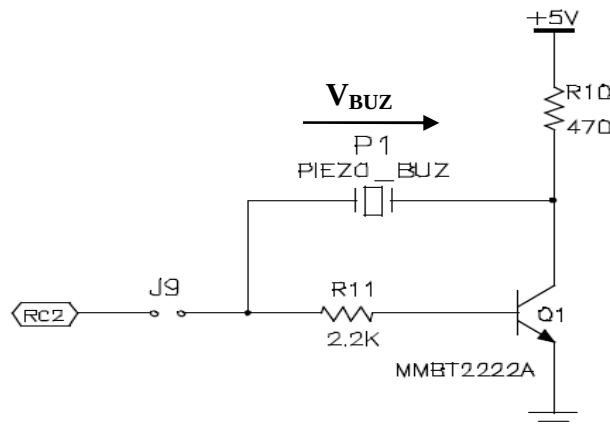


Figure II.1.

Analyse du schéma de la figure II.1. :

Remarquez que c'est la ligne **RC2** du **PORTC** (si le jumper **J9** est mis : à vérifier !!!) qui permet de commander le buzzer **P1**.

Si on suppose qu'on envoie sur la broche de sortie **RC2** un signal carré **TTL** de fréquence f_0 (et de période $T_0 = \frac{1}{f_0}$), quelle est la forme de la tension V_{BUZ} aux bornes du cristal piézo-électrique **P1** ?

Que se passera t'il alors ?

Sur quelle plage de la fréquence f_0 , le son émis sera-t-il audible ?

Pour la suite on prendra $f_0 = 500 \text{ Hz}$.

Pour tester ce phénomène, écrire en C deux fonctions dont les prototypes sont les suivants :

- void **buzzer_off** (void) ; **pas de son émis**, buzzer silencieux.
; par exemple si **RC2 = 0**
- void **buzzer_on** (void) ; **émission** d'un son de fréquence f_0 .
; il faut un signal carré **0 – 5V** sur **RC2**
; un niveau constant **RC2 = 1** ne suffit pas !!!

Tester ces deux fonctions avant de passer à la suite.

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

L'idée de l'application sous interruption par appui sur **S3** à écrire est la suivante :

- au départ le buzzer **P1** est silencieux,
- il faut appuyer **5 fois** sur **S3** pour lui faire émettre un son à **500 Hz**,
- puis à nouveau **3 fois** sur **S3** pour le remettre au silence,
- et ainsi de suite ...

(pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File Import... et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
le fichier : **buzzer it S3.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger Program**), et lancer ce programme...).

Je vous fournis les points importants suivants pour vous aider :

- lire attentivement l'extrait du datasheet suivant de la figure II.2., notamment en ce qui concerne la valeur du bit **INTEDG** du registre SFR **OPTION_REG** :

External interrupt on the RB0/INT pin is edge triggered, either rising, if bit INTEDG (OPTION_REG<6>) is set, or falling, if the INTEDG bit is clear. When a valid edge appears on the RB0/INT pin, flag bit INTF (INTCON<1>) is set. This interrupt can be disabled by clearing enable bit INTE (INTCON<4>). Flag bit INTF must be cleared in software in the Interrupt Service Routine before re-enabling this interrupt. The INT interrupt can wake-up the processor from SLEEP, if bit INTE was set prior to going into SLEEP. The status of global interrupt enable bit, GIE, decides whether or not the processor branches to the interrupt vector following wake-up. See Section 12.13 for details on SLEEP mode.

Figure II.2.

- ne pas oublier d'écrire la routine d'interruption ou **routine d'IT** (qui sera exécutée à chaque appui de **S3**).
(on rappelle qu'une routine d'IT est déclarée un peu comme une fonction mais en utilisant le type **interrupt**. Pourquoi ?)
- ne pas oublier d'acquitter l'interruption (voir bit **INTF**) dans cette routine d'IT,
- ne pas oublier de choisir un **front descendant** pour déclencher l'IT (voir bit **INTEDG**),
- ne pas oublier de valider les interruptions (voir bits **INTE** et **GIE**),
- ne pas oublier d'initialiser le sens des lignes du **PORTB (RB0)** et **PORTC (RC2)** utilisées.

Au final, beaucoup de ces points ont été intégrés dans le canevas de la figure VIII.35. du photocopié de cours complet.

N'hésitez donc pas à vous en inspirer !!!

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

IV. ACCES A L'EEPROM DE DONNEES

Remettre en place le cavalier **J6** et enlever le cavalier **J9**.

Avant de démarrer cette partie, il est important d'avoir pris connaissance des points décrivant **les accès en lecture et en écriture dans l'EEPROM de données** du PIC.

Voir pages 57 à 68 du fichier Résumé du cours pendant TP PIC.

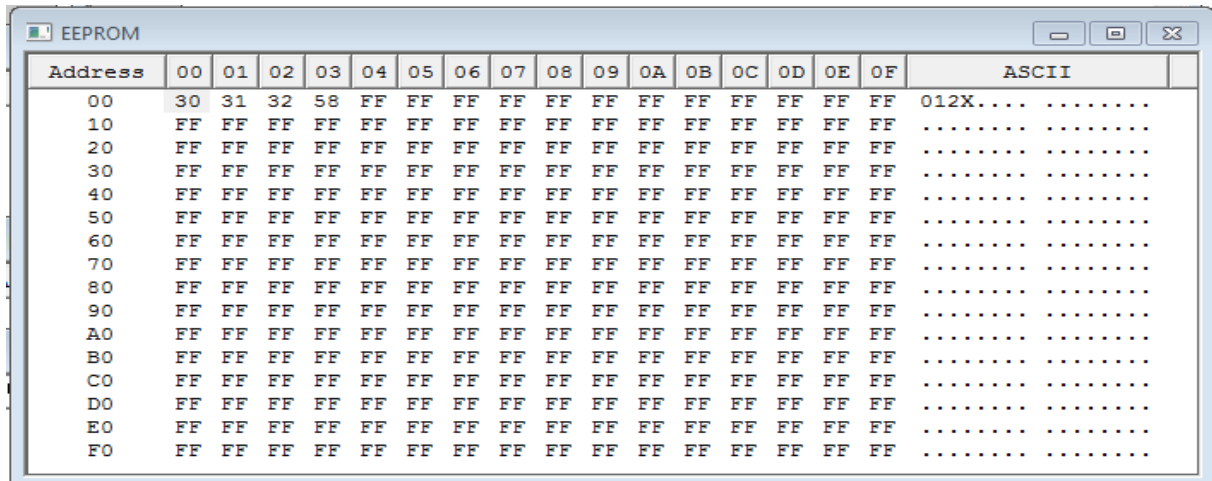
**A la demande des étudiants,
l'enseignant pourra faire une présentation orale de cette partie.**

Remarque importante à lire avant de commencer:

Pour visualiser le contenu de l'EEPROM de données sous **MPLAB**, il faut faire :

View EEPROM

pour ouvrir une fenêtre permettant d'afficher le contenu de l'EEPROM de données, comme celle indiquée sur la figure II.3.



Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
00	30	31	32	58	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	012X....
10	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
20	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
30	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
40	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
50	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
60	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
70	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
80	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
90	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
A0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
B0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
C0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
D0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
E0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
F0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	

Figure II.3.

**Mais attention, lorsqu'on modifie le contenu de l'EEPROM,
cette fenêtre (et idem pour la fenêtre Program Memory)
n'est pas automatiquement remise à jour.**

Il faut pour cela utiliser l'icône suivant, fourni sur la figure II.4.

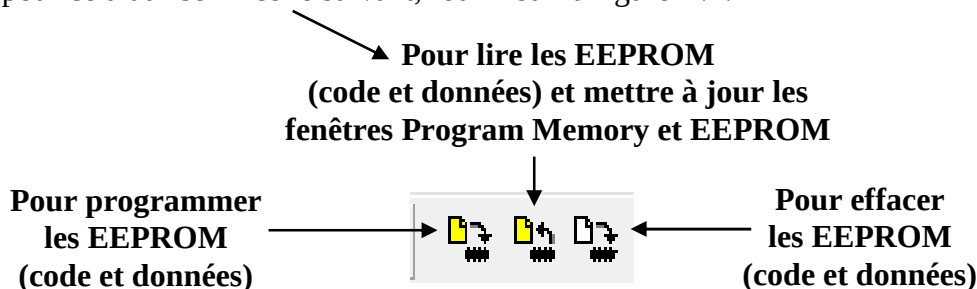


Figure II.4.

On vous demande, dans un premier temps, d'écrire deux fonctions :

- la première **écrit une valeur** dans une case mémoire de **l'EEPROM de données**.

Son prototype est donc le suivant :

void **write_data_eeprom** (char offset, char valeur)

Cette fonction a été détaillée et écrite en **assembleur** aux pages **66** et **67** du **fichier Résumé du cours pendant TP PIC**. On vous demande maintenant de l'écrire en C.

- la seconde **lit une valeur** dans une case mémoire de **l'EEPROM de données**.

Son prototype est donc le suivant :

char **read_data_eeprom** (char offset)

Cette fonction a été détaillée et écrite en **assembleur** à la page **63** du **fichier Résumé du cours pendant TP PIC**. On vous demande maintenant de l'écrire en C.

Testez vos deux fonctions avant de les utiliser dans l'une des applications suivantes.

Applications : réaliser, au choix, une des 2 applications suivantes :

① **Ecrire** et **tester** une fonction de prototype :

char **write_and_verify_data_eeprom** (char offset, char valeur)

dont le cahier des charges est à la page **68** du **fichier Résumé du cours pendant TP PIC**.

② On dispose d'un code pin constitué de 4 chiffres de votre choix, déclarés et initialisés en RAM dans un tableau nommé **code_pin**. On déclarera et initialisera également un second tableau **buffer_clavier** constitué des 4 chiffres tapés au clavier par un utilisateur. Ecrire une fonction de prototype :

void **write_code_pin** (char *s)

qui écrit les 4 chiffres du code pin dans l'EEPROM de données à l'adresse de votre choix. (son paramètre sera donc le tableau **code_pin**). Tester cette fonction.

Puis, à la suite, écrire une autre fonction de prototype :

char **verify_code_pin** (char *s)

(son paramètre sera donc le tableau **buffer_clavier**) qui renvoie :

- soit **0x01**, si le code pin tapé au clavier est **OK**,
- soit **0x00**, si le code pin tapé au clavier est **KO**.

On pourrait aussi envisager d'**effacer** le code pin au bout de **3 essais infructueux !!!**

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

V. UTILISATION DU CONVERTISSEUR ANALOGIQUE/NUMERIQUE

Avant de démarrer cette partie, il est important d'avoir pris connaissance des points essentiels décrivant **le Convertisseur Analogique/Numérique (CAN)** du PIC.

Voir pages 69 à 79 du fichier Résumé du cours pendant TP PIC.

**A la demande des étudiants,
l'enseignant pourra faire une présentation orale de cette partie.**

Important pour démarrer : toujours un œil sur le hardware !!!

Prenez connaissance du schéma de la carte **PICDEM 2 PLUS** (dans le répertoire **documentations...**) et repérer la partie concernée ici notamment pour savoir **sur quel PORT** et **sur quelle ligne de PORT** est injectée la tension analogique à convertir.
Comment peut-on modifier cette tension analogique envoyée sur le PIC?

De plus, vous choisirez ici de travailler avec la précision minimum du CAN et avec une plage de conversion entre VSS (0 V) et VDD (5V).

En utilisant ces résultats, on réalisera ensuite en langage C une fonction de prototype :

```
void  init_can (void) ;
```

permettant d'initialiser correctement le CAN en fonction de son environnement sur la carte de développement et des contraintes précédentes.

Puis on réalisera une fonction de conversion fonctionnant par pooling sur le bit **GO/DONE** (reconnu par le compilateur sous le nom de **GODONE** ou **ADGO** : voir pour s'en convaincre le fichier **pic16x8a.h** attaché à votre projet), qui renverra le résultat d'une conversion effectuée par le CAN et dont le prototype est le suivant :

```
unsigned char conversion (void)
```

Aide et conseil à suivre avant de démarrer :

Comme cela a été vu, penser à mettre une temporisation d'environ 14 µs pour assurer l'acquisition du signal analogique. La durée de cette temporisation pourra avantageusement être testée sous le simulateur de **MPLAB** avec une fenêtre **StopWatch**.

Avant d'utiliser ces fonctions dans l'application suivante, vous les testerez dans un main sur la carte **PICDEM 2 PLUS** pour plusieurs positions du potentiomètre **RA0**.

Réaliser une des deux applications suivantes : à ne faire **impérativement** que lorsque les deux fonctions précédentes auront été testées !!!

Application 1 :

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File Import... et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
le fichier : **sifflet.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger Program**), vérifier que le jumper **J9** est
bien en place, lancer ce programme et jouer sur le potentiomètre **RA0...**

A vous de faire les bons choix pour obtenir un résultat comparable. Si vous n'avez pas d'idée pour les fréquences à générer et **sous contrôle de l'enseignant**, vous pouvez visualiser sur un oscilloscope et en temps réel la tension sur **RC2**.

Application 2 :

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File Import... et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
le fichier : **can.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger Program**), lancer ce programme et jouer
sur le potentiomètre **RA0...**

Réaliser cette petite application qui pilote les leds **D2, D3, D4** et **D5** en fonction de la tension analogique injectée : une sorte de compteur binaire de 0 à 15, lorsque la tension analogique varie de 0V à 5V.

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!
--

Fin de TP N°2 :

**Et n'oubliez pas de préparer le TP N°3 pour la prochaine séance.
(cf. page de garde du TP N°3) !!!**

TP N°3

Objectifs principaux :

Utilisation des timers internes par scrutation et/ou interruption
Prise en main du LCD
Mise en œuvre du LCD
Utilisation des modules CCP1 et CCP2
Fonction CAPTURE : mesure de périodes et/ou de fréquences (tests)
Fonction COMPARE : générations de sons (tests)
Génération de signal PWM
Signal PWM commandé par une consigne

Sur le polycopié de cours complet :

Lire chapitre VIII.5.1. Généralités sur la fonction timer

Lire chapitre VIII.5.2.1. Le TIMER0

Lire chapitre VIII.5.2.2. Le TIMER1

Sur le polycopié du TP N°3 :

Partie I sur le TIMER0 : répondre aux questions 1, 2, 3 et 4.

Partie II sur le TIMER1 : répondre aux questions 1, 2, 3 et 4.

Lire et bien comprendre sur le polycopié du TP N°3

Etape 1 à étape 6 de la partie III sur le LCD

*Là aussi, si cette préparation n'est pas faite,
aucune chance d'avancer loin dans ce TP N°3 !!!*

N'oubliez pas que je vérifie votre préparation en début de TP !!!

Enseignant : THIEBAUT Bernard

Tous les conseils de programmation fournis dans les précédents TP restent bien sûr à l'ordre du jour.

I. UTILISATION DU TIMER0 PAR SCRUTATION

Avant de démarrer cette partie, il est important d'avoir pris connaissance des points essentiels sur les **généralités** de la fonction **timer** puis ensuite les **spécificités** du **TIMER0** du PIC.

Voir pages 80 à 94 du fichier Résumé du cours pendant TP PIC.

**A la demande des étudiants,
l'enseignant pourra faire une présentation orale de cette partie.**

1. Quelle est la valeur $T0_{max}$ de la temporisation la plus grande que l'on peut réaliser avec le **TIMER0 ?**

Est-ce suffisant pour réaliser une temporisation $T0_1$ de 1 seconde avec le **TIMER0 ?**

2. En déduire une méthode pour réaliser cette temporisation $T0_1$ de 1 seconde.

Aide :

On pourra décomposer $T0_1$ sous la forme (division entière) :

$$T0_1 = k.T0_{max} + T0_R \text{ avec } k \text{ entier et } T0_R < T0_{max},$$

(aucun arrondi sur $T0_{max}$ et sur $T0_R$ ne doit être fait dans les calculs sous peine de perdre de la précision)

3. Mettre en pratique votre méthode pour réaliser une temporisation de 1 seconde.

La technique utilisée sera par scrutation (pooling) et on écrira une fonction C dont le prototype est le suivant :

`void            tempo_1_s    (void) ;`

Remarque :

Pour tester et calibrer correctement votre fonction **tempo_1_s**, vous pouvez dans un premier temps travailler avec le simulateur de **MPLAB** et utiliser une fenêtre **StopWatch...**

Est-il optimal, pour la précision de la fonction **tempo_1_s**, de travailler comme ici en utilisant la valeur maximum du pré diviseur programmable p ? Détailler correctement votre réponse et proposer une solution théorique (ou tout du moins quelques éléments de réponse) pour obtenir la précision la plus grande possible.

4. Puis, en utilisant cette fonction **tempo_1_s, écrire la fonction C suivante :**

`void            tempo_N_s    (int    N);`

qui réalise une temporisation de **N secondes**.

5. Petite application

Et enfin, faire **clignoter** la led **D2** au rythme de **4 secondes allumée, 2 secondes éteinte**. (en utilisant cela va de soi ; la fonction **tempo_N_s** écrite précédemment !!!)

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

II. UTILISATION DU TIMER1 SOUS INTERRUPTION

Avant de démarrer cette partie, il est important d'avoir pris connaissance des points essentiels sur les **spécificités** du **TIMER1** du PIC.

Voir pages 95 à 99 du fichier **Résumé du cours pendant TP PIC**.

A la demande des étudiants,
l'enseignant pourra faire une présentation orale de cette partie.

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
le fichier : **chrono timer 1.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

On se propose donc de réaliser une « petite montre » qui compte les secondes et les affiche sur les leds **D5** à **D2**.

On tiendra donc à jour une variable globale (de type **unsigned char**) **seconde**, qui sera incrémentée sous l'interruption du **TIMER1**.

1. En utilisant une méthode analogue à l'exercice précédent, calculer la valeur **T1_{max}** de la temporisation **la plus grande** réalisable avec le **TIMER1**.

En conséquence, la routine d'IT ne pourra pas incrémenter la variable seconde à chaque interruption !!! Expliquer pourquoi.

2. En déduire que, pour réaliser une temporisation de 1 seconde, il faut décomposer :

$$1 \text{ seconde} = T1_{\max} + T1_R$$

et donner la valeur **VI** à charger dans le registre compteur **TMR1** du **TIMER1** pour réaliser la temporisation **T1_R**.

En conséquence, il faudra au moins deux interruptions successives avant d'incrémenter la variable seconde, comme indiqué sur le schéma de principe de la figure III.1.

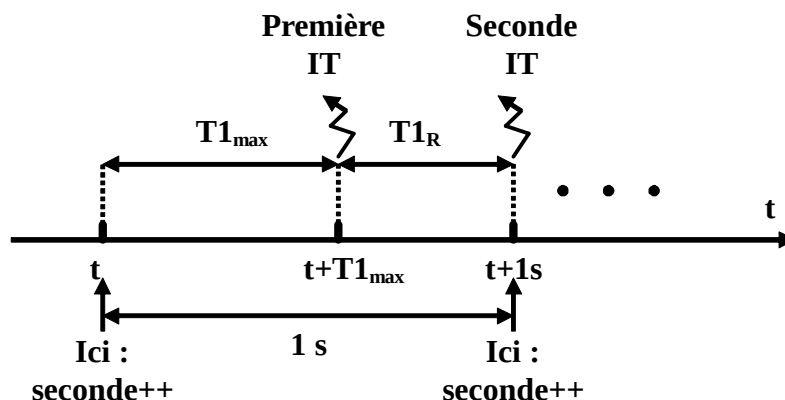


Figure III.1.

3. Ecrire alors la routine d'IT (en prenant en compte la remarque précédente) qui tient à jour la variable globale **seconde**.

4. Mise en application :

Examiner la fonction C **affiche_seconde** de la figure III.2. suivante :

```
void affiche_seconde(unsigned char seconde)
{
    switch(seconde)
    {
        case 0      : PORTB=0xF0;break;
        case 1      : PORTB=0xF1;break;
        case 2      : PORTB=0xF2;break;
        case 3      : PORTB=0xF3;break;
        case 4      : PORTB=0xF4;break;
        case 5      : PORTB=0xF5;break;
        case 6      : PORTB=0xF6;break;
        case 7      : PORTB=0xF7;break;
        case 8      : PORTB=0xF8;break;
        case 9      : PORTB=0xF9;break;
        case 10     : PORTB=0xFA;break;
        case 11     : PORTB=0xFB;break;
        case 12     : PORTB=0xFC;break;
        case 13     : PORTB=0xFD;break;
        case 14     : PORTB=0xFE;break;
        case 15     : PORTB=0xFF;break;
    }
}
```

Figure III.2.

Que réalise cette fonction ?

Proposer un codage plus concis pour réaliser la même chose.

Réaliser enfin un main qui affiche (et compte) les secondes indéfiniment sur les leds **D5** à **D2**. (en utilisant cela va de soi ; la fonction **affiche_seconde** et la routine d'interruption du **TIMER1** écrites précédemment !!!)

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

III. PRISE EN MAIN DU LCD DE LA CARTE PICDEM2 PLUS

A la demande des étudiants,
l'enseignant pourra faire une présentation orale de cette partie.

Voir pages 132 à 143 du fichier Résumé du cours pendant TP PIC.

Pour avoir une idée sur ce qu'on demande de réaliser avec le LCD, sous MPLAB faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
le fichier : **test lcd.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

Jusqu'à maintenant, nous n'avons utilisé que les leds (ou exceptionnellement le buzzer) pour effectuer des sorties sur la carte **PICDEM2 PLUS**.

C'est assez limitant, mais c'était une étape nécessaire pour l'apprentissage progressif que vous venez de réaliser.

Or, il existe un **LCD** (avec 2 lignes de 16 caractères chacune) sur cette carte, et nous allons apprendre ici à le mettre en œuvre.

Il nous servira ensuite pour la suite des exercices de ce TP.

Mais, il est plus compliqué d'utiliser un **LCD** qu'une simple led.

Du coup, je ne demanderai pas de tout écrire, mais plutôt de **tout comprendre !!!**

Inutile ici de faire un nouveau projet car je fournis un exemple de projet **test lcd.mcp** « incomplet » qu'il faudra compléter à certains endroits « stratégiques » pour pouvoir utiliser le **LCD**.

Ouvrez donc le projet **test lcd.mcp** qui se trouve dans le sous répertoire **prog_c** de **votre répertoire** de travail.

Par contre, la méthode que l'on va utiliser pour implanter ce périphérique sera volontairement très guidée : cela vous permettra de comprendre les différentes étapes qui sont nécessaires pour mettre en œuvre une nouvelle interface.

Méthode que vous pourrez reproduire « dans l'esprit » si vous devez par vous-même implanter votre propre périphérique :

- module de positionnement GPS,
- écran graphique,
- caméra,
- modules ultra-sons pour mesurer des distances,
- modules de reconnaissance vocale,
- ou de façon générale, tout autre type de capteur ou d'actionneur ...

② les 8 broches **d'entrée/sortie DB0 à DB7** permettent d'échanger les informations (contenu de **IR** et de **DR**) avec l'extérieur.

③ les pattes d'entrées **RS** (comme **Register Select**) et **R/ $\overline{W}$** (comme **Read/ $\overline{\text{Write}}$**) du **LCD** permettent d'accéder à ces deux registres internes, conformément au tableau donné sur la figure III.4. :

$\text{R/} \overline{\text{W}}$ \ RS	0	1
0	Écriture dans IR	Écriture dans DR
1	Lecture de IR	Lecture de DR

Figure III.4.

④ **après avoir positionné** les entrées **RS** et **R/ $\overline{W}$** du **LCD** suivant ce que l'on veut faire, il faut envoyer un « pulse » sur l'entrée **E** de la forme fournie sur la figure III.5.

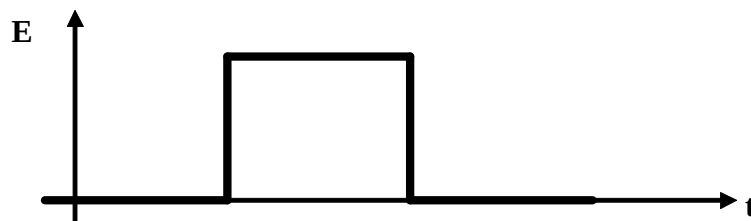


Figure III.5.

En toute rigueur, certaines contraintes sont à respecter pour ce signal, mais en programmant en C et compte tenu de la fréquence de 4 MHz du quartz utilisé, elles seront toujours respectées.

Un exemple de chronogramme d'accès au **LCD**, issu de son datasheet est d'ailleurs fourni sur la figure III.6., pour le cas d'une écriture (pour plus d'informations, consultez les datasheet)

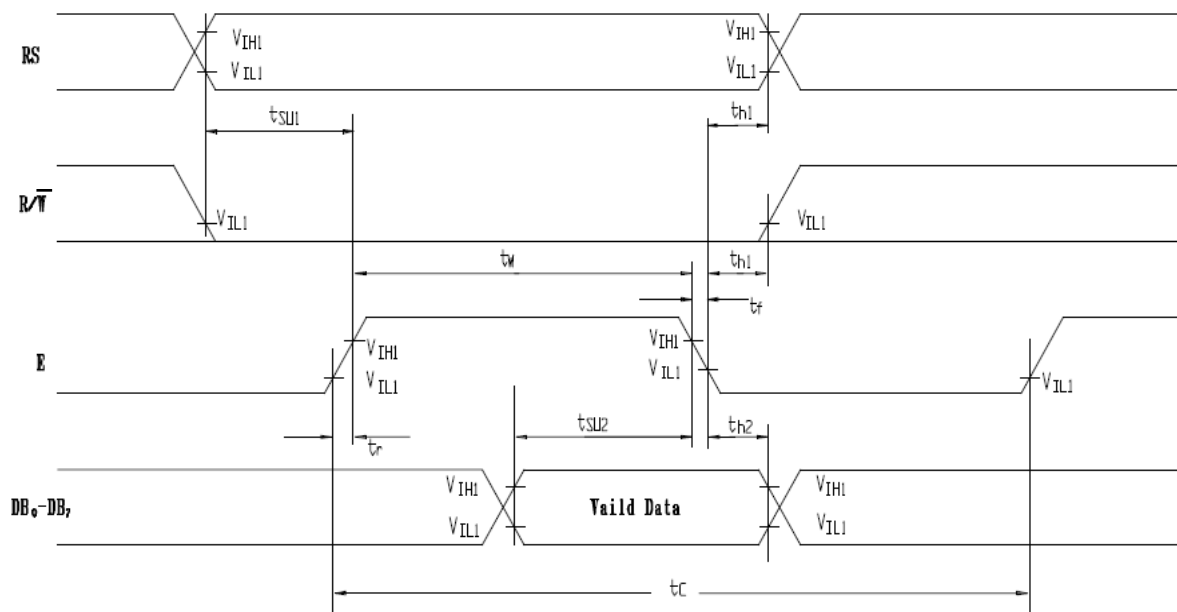


Figure III.6.

- ⑤ la patte d'entrée **VEE** du **LCD** permet de régler le contraste,
- ⑥ les pattes d'entrée **VCC** et **GND** servent à l'alimentation du **LCD**,

Voilà, on en sait assez pour passer à l'étape suivante !!!

Etape 2 : lecture du schéma d'implantation du LCD sur la carte PICDEM 2 PLUS et affectation des broches

Le schéma d'implantation du LCD est fourni sur la figure III.7.

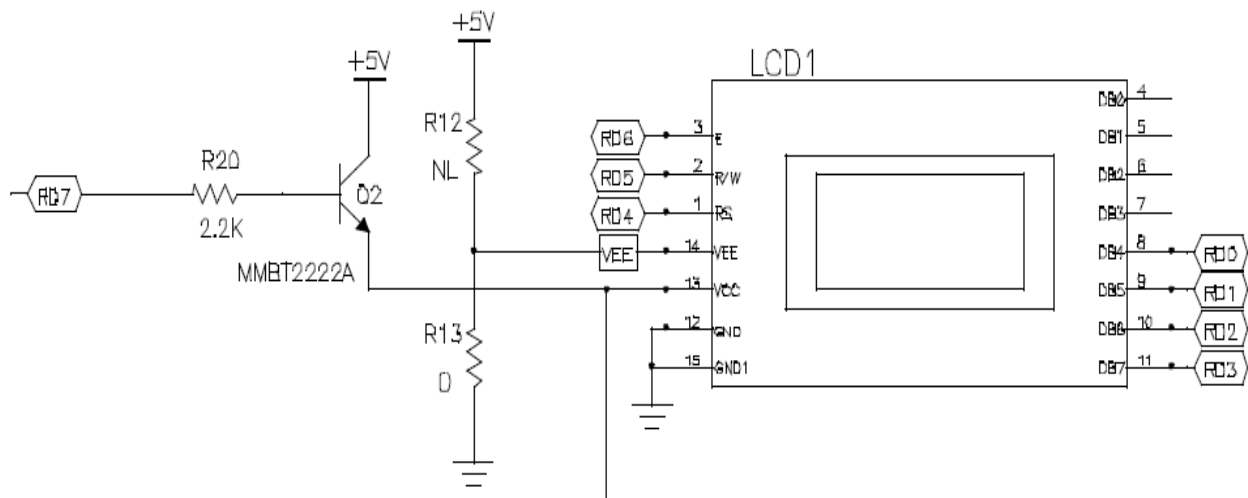


Figure III.7.

Cette étape permet de faire le lien entre le **hardware** et les **noms symboliques** qui seront utilisés par le programme pour les différentes broches, ceci pour améliorer la lisibilité et la compréhension du source en C.

Il faut donc maintenant faire les correspondances entre ce schéma et ce que vous avez compris dans l'étape précédente.

1. Compléter les #define du fichier **lcdbt.h** attaché au projet **test lcd.mcp**. (certains existent, continuer avec la même rigueur)

Pourquoi met-on ces affectations dans un fichier de type .h ?

Ces noms symboliques seront à utiliser **obligatoirement** dans la suite.

A partir de cet instant, votre projet **test lcd.mcp** est « compilable » (**à vérifier impérativement avant de continuer**) mais encore incomplet...

2. Ecrire le **corps** de la fonction :

```
void init_PORTD (void)
```

permettant de choisir les directions des différentes lignes du **PORTD**. Utiliser bien sûr les noms symboliques précédemment choisis.

Cette fonction se trouve dans le fichier **lcdbt.c** attaché aussi au projet **test lcd.mcp**.

(les lignes **RD0** à **RD3** étant en entrée/sortie, leur direction sera choisie dynamiquement au moment où elles seront utilisées dans le programme).

(ce n'est pas difficile, ne cherchez pas midi à quatorze heures !!!)

Pourquoi serait-il plus structurant d'intégrer cette fonction dans la fonction **allume_LCD** (voir plus loin à quoi sert **allume_LCD**). **Faites-le.**

3. Quelle tension faut-il appliquer sur la patte **RD7** du **PORTD** pour alimenter le **LCD** ?

Ecrire le **corps** de la fonction :

```
void allume_LCD (void)
```

permettant d'allumer le **LCD**.

Utiliser toujours les noms symboliques précédemment choisis.

Cette fonction se trouve aussi dans le fichier **lcdbt.c** attaché au projet test **lcd.mcp**.

(là aussi, ce n'est pas difficile, ne cherchez pas midi à quatorze heures !!!)

4. N'est-il pas étonnant de n'avoir câblé que les 4 broches **DB4** à **DB7** du **LCD** alors que les registres **IR** et **DR** sont tous les deux de taille 8 bits ?
(des éléments de réponse à cette question seront fournis à l'étape suivante)

Etape 3 : initialisation du LCD

Il faut respecter toute une procédure préconisée par le constructeur et que l'on trouve dans la documentation du **LCD**. Elle est reprise sur la figure III.8.

C'est la partie la plus délicate car il faut respecter des temporisations, liées au fait que le **LCD** met un certain temps pour exécuter chaque instruction d'initialisation : du coup, si on envoie une autre instruction alors que la précédente n'est pas terminée, ça ne fonctionnera pas...

J'ai importé pour cela dans le projet une fonction de temporisation **paramétrable** :

```
void tempo_N_ms (int N)
```

utilisant le **TIMER0** en pooling.

De plus, on travaille ici en mode 4 bits, ce qui implique que les accès (en lecture ou en écriture) vers le **LCD** se feront en deux temps :

- **d'abord** le quartet de poids **fort**,
- et **ensuite** le quartet de poids **faible**.

Vérifier que la fonction :

```
void init_LCD (void)
```

que j'ai écrite pour vous est conforme avec l'organigramme d'initialisation de la figure III.8.

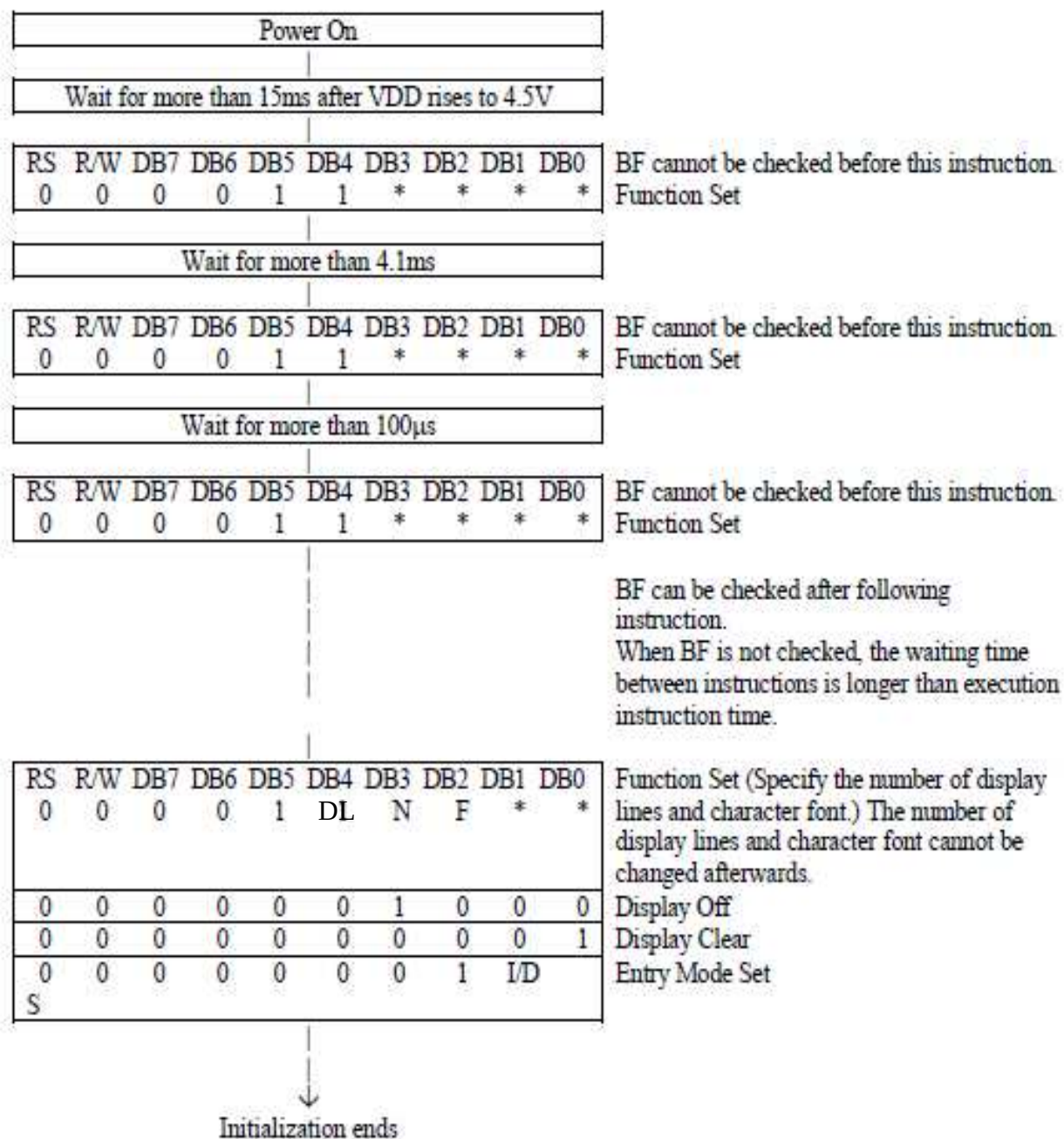


Figure III.8.

Pour effectuer la correspondance entre la fonction **init_LCD** et cet organigramme, vous devrez parfois aussi vous reporter au tableau des commandes reconnues par le **LCD** et fournies sur la figure III.9., reprise du datasheet du **LCD**.

Command	RS	R/W	DB ₇	DB ₆	DB ₅	DB ₄	DB ₃	DB ₂	DB ₁	DB ₀	Execution Time (f _{osc} = 250kHz)	Remark																		
DISPLAY CLEAR	L	L	L	L	L	L	L	L	L	H	1.64ms																			
RETURN HOME	L	L	L	L	L	L	L	L	H	X	1.64ms	Cursor move to first digit																		
ENTRY MODE SET	L	L	L	L	L	L	L	H	I/D	SH	42μs	- I/D : Set cursor move direction<table><tr><td>I/D</td><td>H</td><td>Increase</td></tr><tr><td></td><td>L</td><td>Decrease</td></tr></table> - SH : Specifies shift of display<table><tr><td>SH</td><td>H</td><td>Display is shifted</td></tr><tr><td></td><td>L</td><td>Display is not shifted</td></tr></table>	I/D	H	Increase		L	Decrease	SH	H	Display is shifted		L	Display is not shifted						
I/D	H	Increase																												
	L	Decrease																												
SH	H	Display is shifted																												
	L	Display is not shifted																												
DISPLAY ON/OFF	L	L	L	L	L	L	H	D	C	B	42μs	- Display<table><tr><td>D</td><td>H</td><td>Display on</td></tr><tr><td></td><td>L</td><td>Display off</td></tr></table> - Cursor<table><tr><td>C</td><td>H</td><td>Cursor on</td></tr><tr><td></td><td>L</td><td>Cursor off</td></tr></table> - Blinking<table><tr><td>B</td><td>H</td><td>Blinking on</td></tr><tr><td></td><td>L</td><td>Blinking off</td></tr></table>	D	H	Display on		L	Display off	C	H	Cursor on		L	Cursor off	B	H	Blinking on		L	Blinking off
D	H	Display on																												
	L	Display off																												
C	H	Cursor on																												
	L	Cursor off																												
B	H	Blinking on																												
	L	Blinking off																												
SHIFT	L	L	L	L	L	H	S/C	R/L	X	X	42μs	<table><tr><td>S/C</td><td>H</td><td>Display shift</td></tr><tr><td></td><td>L</td><td>Cursor move</td></tr></table> <table><tr><td>R/L</td><td>H</td><td>Right shift</td></tr><tr><td></td><td>L</td><td>Left shift</td></tr></table>	S/C	H	Display shift		L	Cursor move	R/L	H	Right shift		L	Left shift						
S/C	H	Display shift																												
	L	Cursor move																												
R/L	H	Right shift																												
	L	Left shift																												
SET FUNCTION	L	L	L	L	H	DL	N	F	X	X	42μs	<table><tr><td>DL</td><td>H</td><td>8 bits interface</td></tr><tr><td></td><td>L</td><td>4 bits interface</td></tr></table> <table><tr><td>N</td><td>H</td><td>2 line display</td></tr><tr><td></td><td>L</td><td>1 line display</td></tr></table> <table><tr><td>F</td><td>H</td><td>5 X 10 dots</td></tr><tr><td></td><td>L</td><td>5 X 7 dots</td></tr></table>	DL	H	8 bits interface		L	4 bits interface	N	H	2 line display		L	1 line display	F	H	5 X 10 dots		L	5 X 7 dots
DL	H	8 bits interface																												
	L	4 bits interface																												
N	H	2 line display																												
	L	1 line display																												
F	H	5 X 10 dots																												
	L	5 X 7 dots																												
SET CG RAM ADDRESS	L	L	L	H	CG RAM address (corresponds to cursor address)						42μs	CG RAM Data is sent and received after this setting																		
SET DD RAM ADDRESS	L	L	H	DD RAM address						42μs	DD RAM Data is sent and received after this setting																			
READ BUSY FLAG & ADDRESS	L	H	BF	Address Counter used for both DD & CG RAM address						0μs	<table><tr><td>BF</td><td>H</td><td>Busy</td></tr><tr><td></td><td>L</td><td>Ready</td></tr></table> – Reads BF indication internal operating is being performed – Reads address counter contents	BF	H	Busy		L	Ready													
BF	H	Busy																												
	L	Ready																												
WRITE DATA	H	L	Write Data						46μs	Write data into DD or CG RAM																				
READ DATA	H	H	Read Data						46μs	Read data from DD or CG RAM																				

X : Don't care

Figure III.9.

Etape 4 : fonction d'écriture d'un caractère sur le LCD

En vous inspirant du code de la fonction :

```
void ecriture_commande (char)
```

qui envoie une instruction dans le registre **IR** du **LCD**, écrire le **corps** de la fonction :

```
void print_char (char)
```

permettant d'envoyer (et donc d'afficher) un caractère dans le registre **DR** du **LCD**.

Cette fonction se trouve aussi dans le fichier **lcdbt.c** attaché au projet test **lcd.mcp**.

Aide : essayer de bien comprendre l'utilité du **ou logique** avec le masque **0x80** dans les lignes suivantes de la fonction **ecriture_commande** :

```
LCD_PORT=((commande>>4)&0x0F)|0x80;    //poids fort commande et maintien VCC
```

```
LCD_PORT=(commande&0x0F)|0x80;    //poids faible commande et maintien VCC
```

Tester cette fonction dans le main du projet **test lcd.mcp** **avant de passer** à l'étape suivante.

Etape 5 : fonction d'écriture d'une chaîne de caractères sur le LCD

En utilisant la fonction **print_char** précédente (programmation modulaire), écrire le **corps** de la fonction :

```
void print_string (char *)
```

permettant d'envoyer (et donc d'afficher) une chaîne de caractères sur le **LCD**.

(l'argument passé à cette fonction est un **pointeur** sur la chaîne à afficher).

Cette fonction se trouve aussi dans le fichier **lcdbt.c** attaché au projet test **lcd.mcp**.

Tester cette fonction dans le main du projet **test lcd.mcp** avant de passer à l'étape suivante.

Etape 6 : fonctions de gestion du curseur

Les adresses mémoire dans le **LCD** des différentes positions des caractères sont données sur la figure III.10.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	← DISPLAY POSITION
FIRST LINE	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	← DD RAM ADDRESS
SECOND LINE	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F	

Figure III.10.

En utilisant le tableau des commandes de la figure III.9., écrire le **corps** des fonctions suivantes :

- utilisation de la commande **SET DDRAM ADDRESS** permettant de choisir la position d’affichage sur l’écran LCD :

```
void goto_ligne_1 (void);      // place le curseur au début de la ligne 1 du LCD
void goto_ligne_2 (void);      // place le curseur au début de la ligne 2 du LCD
void go_to (int);              // place le curseur sur la position x (entier de 1 à 32)
                                // (x=1 à 16 : ligne 1 et x=17 à 32 : ligne 2)
```

- utilisation de la commande **SHIFT** pour décaler le curseur ou l’écran :

```
void curseur_droite (void);    // décale le curseur d’une position à droite
void curseur_gauche (void);    // décale le curseur d’une position à gauche
void ecran_droite (void);      // décale l’écran complet sur la droite
void ecran_gauche (void);      // décale l’écran complet sur la gauche
```

- utilisation de la commande **DISPLAY CLEAR** pour effacer l’écran :

```
void efface (void);            // efface complètement le LCD
```

Ces fonctions se trouvent dans le fichier **lcdbt.c** attaché au projet test **lcd.mcp**.

Tester ces fonctions dans le main du projet **test lcd.mcp**.

Vous disposez maintenant d’un ensemble de fonctions qui vous permettront d’utiliser pleinement le **LCD** de la carte **PICDEM 2 PLUS** !!!!

On pourrait aussi en rajouter d’autres, suivant les besoins...

ON APPELERA L’ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

IV. UTILISATION DU LCD DE LA CARTE PICDEM2 PLUS

Je vous propose trois petites applications permettant de mettre en œuvre le **LCD**.

Pour chaque d'entre elles, beaucoup d'éléments ont déjà été écrits dans les TP précédents.

A vous de faire maintenant du « LEGO » pour assembler les différentes pièces !!!

Dans chaque cas, vous construirez un nouveau projet, comme on l'a toujours fait.

**Essayer de structurer au mieux vos projets
en créant des fichiers .c et .h que vous attacherez à vos différents projets.**

IV.1. MILLIVOLTMETRE/VOLTMETRE

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
 le fichier : **millivoltmetre.hex** ou **voltmetre.hex** (ce sont des exécutables)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), lancer ce programme et régler
le potentiomètre **RA0...**

Utiliser votre travail fait sur le **Convertisseur Analogique Numérique** dans le **TP N°2**.

Pour le format d'affichage de la tension en mV sur le LCD, vous pouvez par exemple utiliser la fonction **sprintf** :

(voir sa documentation dans le manuel du C présent dans le sous répertoire **documentations** de **votre** répertoire de travail)

```
#include <stdio.h>
```

```
int sprintf (char * buf, const char * fmt, ...)
```

IV.2. MESSAGE DEROULANT

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
 le fichier : **message deroulant.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

Pas de commentaire particulier à faire sur ce mini projet !!!

IV.3. MONTRE

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
 le fichier : **montre.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

Aide :

Pour gérer le temps qui s'écoule, et tenir à jour les secondes, minutes et heures, vous pouvez utiliser le **TIMER2** sous interruption grâce au programme présenté dans le polycopié de cours complet sur la figure VIII.91.

Par contre, on vous demande, dans le compte-rendu, d'expliquer en quelques lignes les principes de fonctionnement de ce programme.

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

V. UTILISATION DES MODULES CCP1 ET CCP2

V.1. FONCTION (INPUT) CAPTURE

Avant de démarrer cette partie, il est important d'avoir pris connaissance des points essentiels décrivant la fonction **(INPUT) CAPTURE** du module **CCP1** du PIC.

Voir pages 107 à 115 du fichier Résumé du cours pendant TP PIC.

**A la demande des étudiants,
l'enseignant pourra faire une présentation orale de cette partie.**

Compte tenu du temps, on vous demande **juste de voir** ce qu'on peut faire avec cette fonction **CAPTURE** : un « petit fréquencemètre ».

Réglage préalable :

 : régler un GBF pour qu'il génère un signal **TTL** carré 0 – 5V de fréquence 1 kHz et visualiser ce signal à l'oscilloscope (en mode **DC**) avant de l'utiliser. En effet, si vous envoyez sur la carte PICDEM 2 PLUS des signaux non conformes (tensions négatives ou supérieures à 5V...) **vous risquez fortement de l'endommager définitivement !!!**

Ou bien utiliser la sortie synchro du GBF après avoir fait les réglages précédents...

Pour avoir une idée sur ce qu'on demande de **tester**, sous **MPLAB** faire :

File	Import...	et aller chercher dans votre répertoire de travail
dans le sous répertoire :		fichiers HEX OK/source en C
	le fichier :	frequencemetre.hex (c'est un exécutable)

Retirer le Jumper **J9** pour éviter de faire « couiner » le buzzer.

Injecter ensuite le signal TTL précédemment réglé sur la broche **RC2** du PIC.

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**), envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

Vérifier la plage d'utilisation de ce fréquencemètre et expliquer les limitations obtenues.

V.2. FONCTION (OUPUT) COMPARE

Avant de démarrer cette partie, il est important d'avoir pris connaissance des points essentiels décrivant la fonction **(OUTPUT) CAPTURE** du module **CCP1** du PIC.

Voir pages 116 à 123 du fichier Résumé du cours pendant TP PIC.

**A la demande des étudiants,
l'enseignant pourra faire une présentation orale de cette partie.**

Compte tenu du temps, on vous demande ici **juste de voir** (ou plutôt précisément dans ce cas **d'entendre !!!**) ce qu'on peut faire avec cette fonction **COMPARE** : par exemple une génération de sons sur le buzzer.

Réglage préalable :

Assurer vous que le Jumper **J9** est bien présent et installé et retirer le GBF s'il était encore présent (suite à la manip précédente) sur le broche **RC2** du PIC.

Pour avoir une idée sur ce qu'on demande de tester, sous **MPLAB** faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
le fichier : **son.hex** (c'est un exécutable)

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

On pourra visualiser avec l'oscilloscope le signal présent sur la broche **RC2** du PIC et faire le lien avec le son généré par le buzzer !!!

V.3. FONCTION PWM (PULSE WIDTH MODULATION)

Avant de démarrer cette partie, il est important d'avoir pris connaissance des points essentiels décrivant la fonction **PWM** du module **CCP1** du PIC.

Voir pages 124 à 131 du fichier Résumé du cours pendant TP PIC.

**A la demande des étudiants,
l'enseignant pourra faire une présentation orale de cette partie.**

V.3.1. RAPPORT CYCLIQUE FIXE

Comme précédemment, retirer le Jumper **J9** pour éviter de faire « couiner » le buzzer.

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
le fichier : **pwm fixe.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

Visualiser avec un oscilloscope le signal présent sur la broche **RC2/CCP1** du PIC.

On vous demande de générer sur la broche **RC2/CCP1** du PIC un signal **PWM** dont les caractéristiques sont les suivantes.

- fréquence : $f = 10 \text{ kHz}$,
- rapport cyclique : $\alpha = \frac{1}{3}$.

Les calculs préalables seront détaillés sur votre compte-rendu.

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

V.3.2. RAPPORT CYCLIQUE VARIABLE

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

File **Import...** et aller chercher **dans votre répertoire** de travail
dans le sous répertoire : **fichiers HEX OK/source en C**
 le fichier : **pwm variable.hex** (c'est un exécutable)
Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger** **Select Tool**),
envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

Visualiser avec un oscilloscope le signal présent sur la broche **RC2/CCP1** du PIC et à l'aide du potentiomètre **RA0**, faites varier le rapport cyclique.

L'idée est d'utiliser le potentiomètre **RA0** pour régler le rapport cyclique du signal **PWM** précédent (de fréquence 10 kHz) entre 0 et 100 %.

Pour cela on utilisera le **Convertisseur Analogique Numérique** et les fonctions écrites pour le mettre en œuvre dans le **TP N°2** (initialisation, acquisition, conversion...).

De plus, le rapport cyclique sera affiché en % et en temps réel sur le LCD.

ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!

Voilà, la prise en main des PIC est finie !!!

On pourrait rajouter encore d'autres exercices,
par exemple sur les liaisons séries asynchrones et synchrones ...

J'espère tout de même que ce domaine est maintenant plus clair pour vous
et que vous pourrez mettre en pratique ces connaissances et les approfondir
notamment sur votre projet de robot mobile ou sur des exemples d'autres projets.

Et rappelez-vous :
seule notre imagination est limitative !!!
mais aussi ...
seule une méthodologie rigoureuse et « professionnelle »
associée à une programmation structurée
(avec tests individuels à chaque étape des fonctions)
permettront de réaliser des projets plus ambitieux !!!